

Publicly Verifiable Generalized Secret Sharing Schemes and Their Applications

No Author Given

No Institute Given

Abstract. Secret sharing under expressive access structures and public verifiability have largely been developed in separate lines of work. Generalized secret sharing (GSS) supports arbitrary monotone access structures, but relies on private channels and gives no public evidence that the dealer distributed a consistent sharing. Publicly verifiable secret sharing (PVSS), in contrast, supports public verification over insecure channels, but is fundamentally tailored to threshold policies. We introduce publicly verifiable generalized secret sharing (PVGSS), a primitive that combines arbitrary monotone access structures with non-interactive public verification of both encrypted shares and decrypted reconstruction shares. Our construction encrypts only the leaf shares of an underlying GSS instance and proves, via Schnorr-type NIZK proofs, that all ciphertexts are globally consistent with a single secret. The key technical ingredient is a GSS testing procedure that checks this global consistency from proof responses, without revealing the shares. We instantiate PVGSS from recursive Shamir secret sharing and from linear secret sharing schemes, obtaining verification complexity linear in the number of leaves of the access structure. We formalize correctness, encrypted-share verifiability, share-decryption verifiability, and IND1-secrecy, and prove security in the random oracle model under DDH, knowledge soundness of the proof systems, and privacy of the underlying GSS-on-group sharing. As an application, we show how PVGSS enables publicly accountable secret release in a decentralized exchange protocol.

Keywords: Access structure, Public verifiability, PVGSS, GSS Testing, Decentralized exchange

1 Introduction

Secret sharing (SS) [1] is a basic mechanism for distributing trust: a dealer encodes a secret into shares so that only authorized subsets of parties can reconstruct it. Verifiable secret sharing (VSS) [8,58,59] strengthens this primitive by making dealer misbehavior detectable, and publicly verifiable secret sharing (PVSS) [14,11,10] further allows external observers to verify encrypted shares over public channels. This public-verification feature is essential in settings where there is no trusted delivery channel and where correctness must be checked without interaction.

The limitation is that public verifiability has mostly been developed for threshold access structures. Threshold policies are useful, but they cannot naturally express hierarchical, weighted, compartmented, or composite authorization requirements. Generalized secret sharing (GSS) [15,16,22] addresses this expressiveness problem by supporting arbitrary monotone access structures, and linear secret sharing schemes (LSSS) [7,33,22] underlie many expressive access-control mechanisms, including ciphertext-policy attribute-based encryption (CPABE) [19,20]. Classical GSS and LSSS, however, assume private share delivery and do not provide a public certificate that the dealer generated a globally consistent sharing. Thus, existing primitives leave a gap between expressiveness and public accountability.

This gap is not closed by composing existing PVSS schemes. In a generalized access structure, the shares associated with the leaves are not independent threshold sharings of the same secret; rather, they are recursively or linearly related through the whole access policy. A verifier must therefore check a global statement: the encrypted leaf shares must be encryptions of shares that reconstruct to one common root secret under the specified access structure. Proving only local well-formedness of each ciphertext is insufficient, while revealing the shares would destroy secrecy.

We introduce publicly verifiable generalized secret sharing (PVGSS) to capture this goal. A PVGSS scheme distributes a secret according to an arbitrary monotone access structure, publishes encrypted leaf shares, and provides non-interactive proofs that any party can verify. The dealer proves that the encrypted shares are globally consistent with a single secret; later, each shareholder proves that its decrypted reconstruction share is correct. When PVGSS is used inside a larger protocol, application-specific identifiers can be included by that protocol to prevent cross-session replay.

Our construction starts from a GSS instance, either recursive Shamir secret sharing or an LSSS-based sharing. The dealer encrypts each leaf share as $C_i = \text{pk}_i^{s_i}$ and produces Schnorr-type Fiat-Shamir NIZK proofs for the encrypted shares. The main technical step is a *GSS testing* procedure: using only the NIZK response values, it checks that the hidden shares satisfy the same generalized access structure and bind to the same dealer secret. For reconstruction, shareholders output group shares together with DLEQ proofs, allowing public verification of decryption correctness. Table 1 summarizes how PVGSS relates to existing secret-sharing primitives.

Crucially, PVGSS cannot be obtained by naively combining existing PVSS schemes. The core technical challenge of PVGSS is to prove, in a publicly verifiable manner, that a collection of encrypted shares $\{C_i\}$ is globally consistent with a single underlying secret under an arbitrary access structure. Suppose the access structure is represented as a tree, the secret is associated with the root while encrypted shares are assigned only to the leaf nodes, rendering any straightforward combination of multiple PVSS instances ineffective. To overcome this challenge, we build PVGSS schemes atop the GSS framework, instantiated using either recursive Shamir secret sharing or linear secret sharing schemes (LSSS). Each

Table 1: Comparison of SS-related cryptographic primitives

Scheme	Accountable dealer (or encryptor)	Accountable shareholders (or authorities)	Insecure channel	Public verifiability	Expressiveness
SS	✗	✗	✗	✗	✗
VSS	✓	✗	✗	✗	✗
PVSS	✓	✓	✓	✓	✗
CPABE	✗	✗	✓	✗	✓
LSSS	✗	✗	✗	✗	✓
GSS	✗	✗	✗	✗	✓
PVGSS	✓	✓	✓	✓	✓

- Accountability indicates whether misbehavior by dealers (or encryptors) and shareholders (or authorities) can be detected.
- Insecure channel denotes that secret shares may be distributed over public channels.
- Public verifiability means that correctness can be checked by any external party.
- Expressiveness indicates native support for arbitrary monotone access structures.

GSS share s_i is encrypted under the public key of shareholder P_i and is accompanied by Schnorr-type non-interactive zero-knowledge (NIZK) proofs derived via the Fiat–Shamir heuristic [30]. Furthermore, we propose a verification algorithm (i.e., GSS testing) to enable validity checks for all shares over an arbitrary access structure. With this algorithm, we establish a binding relation between all encrypted shares and the dealer’s secret, thereby addressing the challenge of public verifiability in secret sharing over general access structures.

1.1 Applications

PVGSS introduces a cryptographic primitive that simultaneously supports expressive access control and non-interactive public accountability—two properties that rarely coexist in prior systems. This combination enables new capabilities across several security-critical domains where correctness must be verifiable by untrusted observers and where access policies extend far beyond simple thresholds. In Section 6, we will dive deep into a decentralized exchange (DEX) protocol applying PVGSS.

Large-scale beacon protocols. Existing PVSS-based distributed randomness beacons [3] protocols are fundamentally constrained by threshold policies, limiting committee flexibility and preventing hierarchical or weighted participation. PVGSS enables publicly verifiable share consistency under arbitrary monotone structures, allowing beacon protocols to incorporate multi-tier committees, heterogeneous trust assumptions, and application-specific quorum rules without sacrificing transparency. This expands the design space for scalable, decentralized randomness generation. For example, by partitioning n distributed par-

ties into subgroups of size c , one can define an access control policy of the form (d of ($\frac{c}{3}$ of $P_1, \dots, P_c$), ($\frac{c}{3}$ of $P_{c+1}, \dots, P_{2c}$), $\dots$, ($\frac{c}{3}$ of $P_{(\frac{n}{c}-1)c+1}, \dots, P_n$)), where the parameter d is introduced to balance unpredictability and liveness of the beacon [4,5]. Under this policy, each party communicates only with a small subgroup of peers, thereby improving scalability.

Attribute-based cryptosystem. PVGSS can enhance attribute-based cryptosystems [19,20] by adding public accountability to key distribution and access-control related computations. In particular, PVGSS can be used as an auditable sharing layer for systems that already rely on LSSS-style access policies. Moreover, PVGSS enables CPABE encryptors to outsource encryption to untrusted servers while ensuring public accountability for their behaviors.

Blockchain ecosystems. Blockchain environments require protocols that operate over public channels, tolerate adversarial participants, and provide transparent correctness guarantees. PVGSS naturally applies to the scenarios below:

- PVGSS, functioning as a timed commitment [52], can be effectively used in on-chain **secret escrow** systems [53,54,55] to provide fine-grained and verifiable management of witnesses, thereby facilitating practical deployments of e-voting and auction protocols.
- In blockchain **bridges** [49], PVGSS ensures that the secret values used to lock and release assets are verifiably shared, thereby coordinating data synchronization across multiple validators or blockchains under a single access control policy.
- In **relay networks** [50], PVGSS enables secure forwarding of state proofs with public verifiability among different groups or committees.
- In payment or state **channels** [51], PVGSS supports flexible quorum policies for dispute resolution, allowing participants to reconstruct secrets only under authorized conditions.
- Incorporating PVGSS into **oracle** [48] design strengthens functionality by ensuring decentralized, publicly verifiable data feeds and enabling flexible privilege weighting among oracle nodes.
- By designing robust access control structures, KYC/AML (i.e., the compliance measurements of know your customer/ anti money laundering) can be effectively enforced as required by **stablecoin governance**. For example, an access policy such as (3 of ($Alice, Bob, (t$ of $Reg_1, \dots, Reg_n$))) enables a threshold number of regulators to transparently monitor the transaction between exchangers (i.e., Alice and Bob).

1.2 Contributions

• **PVGSS as an expressive and publicly verifiable primitive.** We formalize Publicly Verifiable Generalized Secret Sharing (PVGSS), which combines arbitrary monotone access structures with non-interactive public verification over insecure channels. The syntax captures setup, sharing, public verification of encrypted shares, publicly verifiable share decryption, and reconstruction. The

security model covers correctness, encrypted-share verifiability, share-decryption verifiability, and IND1-secrecy for unauthorized labelled sets.

- **Constructions via GSS testing.** We build PVGSS from GSS-on-group sharing and instantiate it with two representative generalized sharing mechanisms: recursive Shamir secret sharing and LSSS-based sharing. The dealer encrypts only the leaf shares and attaches Schnorr-type Fiat-Shamir NIZK proofs. The key technical component is *GSS testing*, which uses the proof responses to check global consistency under the declared access structure without revealing the shares. We give both exact-interpolation and dual-code testing variants, and the public verification uses a linear number of group operations in the number of leaves.

- **Security analysis.** We prove the constructions secure in the random oracle model under DDH, knowledge soundness of the Fiat-Shamir proofs, DLEQ soundness, and privacy of the underlying GSS-on-group scheme. Encrypted-share verifiability follows from extraction and GSS-testing soundness; share-decryption verifiability reduces to DLEQ soundness; and IND1-secrecy is established through hybrids that simulate proofs and hide unauthorized shares.

- **A DEX application with implementation evidence.** We apply PVGSS to a decentralized exchange protocol in which exchangers commit to secrets and passive watchers provide accountable arbitration only when disputes arise. We define the ideal DEX functionality and prove that the real-world contract realizes it in the ledger-hybrid model. Because watcher responses are publicly verifiable and economically accountable, the DEX can set $t = 1$ without assuming an honest-watcher threshold for fairness. We implement the PVGSS algorithms in Go and the DEX verification path in Solidity; the appendix reports off-chain benchmark results and on-chain gas costs.

2 Related Works

2.1 Access Structures in Secret Sharing

As is well known, traditional SS, VSS, and PVSS protocols [1,8,14,11,10] are designed around the threshold access structure (TAS). Benaloh et al. [16] introduced the concept of generalized secret sharing (GSS) based on recursive threshold secret sharing, where generalized access structure (GAS) is inherently implied. Beyond TAS- or GAS- based schemes, some other protocols have explored specific types of access control mechanisms.

In particular, Simmons [28] points out that real-world applications require more versatile capabilities for secret sharing. In weighted secret sharing (WSS) [25,17], each participant is assigned a positive weight. Only if the sum of weights of some participants exceeds the predefined threshold value, can the secret be reconstructed.

In multipartite secret sharing (MSS) [26,27], an access structure defined over n participants is partitioned into disjoint $m(\ll n)$ compartments, where participants within the same compartment have equal weights. In MSS, each compartment must achieve a specific internal agreement before contributing to the

reconstruction of the secret. Tassa et al. [27] point out that any GAS can be regarded as a multipartite access structure with singleton compartments. Hence, reconstructing the secret in MSS scheme depends only on the number of participants from each compartment, rather than their individual identities. In this way, practical applications can focus on m compartments, where TAS is applied, instead of n individuals. However, participants may be designed in different access structures in a single system, resulting in different divisions of disjoint compartments.

Hierarchical secret sharing (HSS) [31,32], also referred as multilevel secret sharing [28,31], places participants in disjoint levels according to their importance. HSS can be applied in managing staff in big companies, where employees are assigned to different levels with different privileges. HSS is categorized into disjunctive and conjunctive types [24]. Further, Drăgan et al. [24] introduce the distributive weighted threshold secret sharing, where participants at the same level have the same weight, and the threshold values are 1.

Other related works, such as proactive secret sharing [58] and multiple secret sharing [21], can be regarded as extensions of SS. Moreover, these concepts can also be incorporated into our GSS and PVGSS schemes.

2.2 (Publicly) Verifiable Secret Sharing

Dealer in the SS scheme may distribute invalid shares to deviate from the protocol. To mitigate this problem, VSS [8,13] ensures that participants can verify the validity of the shares distributed by the dealer, while PVSS [14,11] extends this by allowing any external observer to verify the integrity of the shares. Notably, Galil and Yung [61] combine VSS, partition encryption and NIZK proofs to design a more efficient and fully polynomial scheme known as partitioned VSS.

The first VSS scheme, proposed by Feldman [8], enhances Shamir secret sharing by adding public commitments to ensure share validity. Shareholders verify their shares against these commitments, ensuring the dealer's honesty without revealing the secret. Recently, polynomial commitments [12,13,9] are widely adopted to design new VSS schemes. The secret shares of VSS should not be transferred publicly.

Stadler [14] put forward the first PVSS scheme in 1996. Intuitively, PVSS can be achieved by integrating public key encryption into VSS schemes, allowing secret shares to be delivered and verified on public channels. For example, ElGamal, RSA, and Pailler cryptosystems are utilized in [14,37], [36], and [38], respectively. Early PVSS schemes require $O(nt)$ verification complexity, where n and t are the amount of shareholders and threshold value, respectively. In 2017, Cascudo et al. [11] introduced SCRAPE PVSS which for the first time achieves $O(n)$ verification complexity. As the core technique, Reed-Solomon code is employed to make all encrypted shares be verified in a linear operation.

Recently, Cascudo et al. successively proposed ALBATROSS [35], HEP-VSS/DHPVSS [34], and qCLPVSS [10], which all maintain $O(n)$ verification complexity. ALBATROSS [35] leverages low-degree exponent interpolation to

generate Fiat-Shamir NIZK proofs. HEPVSS[34] also uses ElGamal to encrypt the secret share, while DHPVSS[34] leverages Shamir secret sharing on group $\mathbb{G}$ to hide shares. qCLPVSS [10] enables users to recover the original secret $s \in \mathbb{Z}_q$ by leveraging class group, making it applicable in more wider applications, such as distributed key generation [2,10] and MPC [10,34] protocols.

3 Preliminaries

In this paper, $\mathbb{Z}_q$ denotes a finite field with prime order q , and $\mathbb{G}$ represents a cyclic group of the same order generated by g .

3.1 Lagrange Interpolation and Reed-Solomon Codes

We recall two standard algebraic notions used throughout the paper when reasoning about Shamir-type shares and polynomial consistency.

Definition 1 (Lagrange interpolation). Let $\alpha_1, \dots, \alpha_m \in \mathbb{Z}_q^*$ be distinct evaluation points. For an index set $I \subseteq [m]$, the Lagrange basis polynomial associated with $i \in I$ is $\ell_i^I(X) = \prod_{j \in I, j \neq i} \frac{X - \alpha_j}{\alpha_i - \alpha_j}$. Given values $\{y_i\}_{i \in I}$ with $|I| = t$, there is a unique polynomial $f \in \mathbb{Z}_q[X]$ of degree less than t satisfying $f(\alpha_i) = y_i$ for all $i \in I$, namely $f(X) = \sum_{i \in I} y_i \ell_i^I(X)$. In particular, Shamir reconstruction recovers the constant term as $f(0) = \sum_{i \in I} y_i \ell_i^I(0) = \sum_{i \in I} y_i \prod_{j \in I, j \neq i} \frac{-\alpha_j}{\alpha_i - \alpha_j}$. More generally, the same interpolation formula determines $f(\alpha_k)$ at any other evaluation point α_k . Thus a degree-bounded polynomial is uniquely determined by sufficiently many evaluations, and all other evaluations can be derived from them.

Definition 2 (Reed-Solomon code and dual code). For distinct evaluation points $\alpha = (\alpha_1, \dots, \alpha_m) \in (\mathbb{Z}_q^*)^m$, the $[m, t]$ Reed-Solomon code over $\mathbb{Z}_q$ is $\text{RS}_\alpha(m, t) = \{(f(\alpha_1), \dots, f(\alpha_m)) : f \in \mathbb{Z}_q[X], \deg(f) < t\}$. Its dual code is $\text{RS}_\alpha(m, t)^\perp = \{\mathbf{h} \in \mathbb{Z}_q^m : \langle \mathbf{h}, \mathbf{c} \rangle = 0 \text{ for all } \mathbf{c} \in \text{RS}_\alpha(m, t)\}$. Consequently, a vector $\mathbf{y} \in \mathbb{Z}_q^m$ is a valid Reed-Solomon codeword if and only if $\langle \mathbf{h}, \mathbf{y} \rangle = 0$ for every $\mathbf{h} \in \text{RS}_\alpha(m, t)^\perp$. Equivalently, if $\mathbf{y} \notin \text{RS}_\alpha(m, t)$, then its syndrome is nonzero, and a uniformly random linear combination of dual parity checks accepts it with probability exactly $1/q$.

In particular, Shamir share vectors are Reed-Solomon codewords. If the constant term is fixed to a value x , then x and evaluations $y_1, \dots, y_m$ are consistent with a degree- $< t$ polynomial if and only if there exists $f \in \mathbb{Z}_q[X]$ with $\deg(f) < t$, $f(0) = x$, and $f(\alpha_i) = y_i$ for all $i \in [m]$. Equivalently, the extended vector $(x, y_1, \dots, y_m)$ is a Reed-Solomon codeword over the evaluation points $(0, \alpha_1, \dots, \alpha_m)$.

3.2 Generalized Access Structure (GAS)

Definition 3 (GAS [16,20]). Let $\mathcal{P} = \{P_1, \dots, P_n\}$ be a set of parties. A collection $\mathbb{A} \in 2^{\mathcal{P}}$ is a generalized access structure (GAS, commonly abbreviated as AS in previous research [16,20]), if it meets the following two conditions:

- *Non-triviality*: if $B \in \mathbb{A}$, then $|B| \neq 0$.
- *Monotonicity*: if $B \in \mathbb{A}$, $B \subseteq C$, then $C \in \mathbb{A}$.

If $B \in \mathbb{A}$, we say $\mathbb{A}$ is satisfied by B or B is an authorized set; Otherwise, $\mathbb{A}$ is not satisfied by B or B is unauthorized.

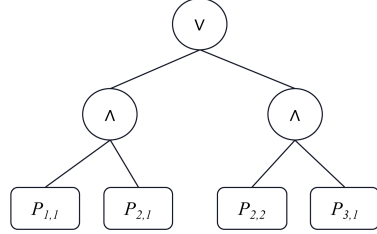


Fig. 1: An access structure using boolean formula, where $\{P_{1,1}, P_{2,1}\}$ and $\{P_{2,*}, P_{3,1}\}$ are authorized sets.

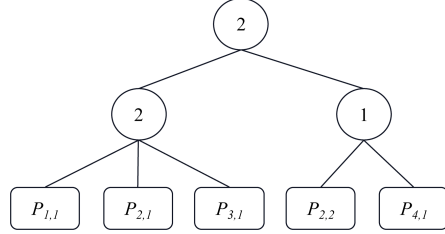


Fig. 2: An access structure using threshold gate, where $\{P_{3,1}, P_{2,*}\}$ and $\{P_{1,1}, P_{3,1}, P_{4,1}\}$ are authorized sets.

For example, $(P_{1,1} \wedge P_{2,1}) \vee (P_{2,2} \wedge P_{3,1})$ and 2 of $(2 \text{ of } (P_{1,1}, P_{2,1}, P_{3,1}), (1 \text{ of } (P_{1,2}, P_{4,1})))$ are access structures, as shown by Figure 1 and Figure 2, where $P_{i,j}$ represents the j th authorization granted to party P_i and $*$ is a wildcard. The disjunction ($\vee$) and conjunction ($\wedge$) in a boolean formula based access structure (cf. Figure 1) is equal to the gate value of 1 and 2 in a threshold-gate based access structure (cf. Figure 2), respectively. Hence, we only consider threshold-gate based access structure in this paper.

Below, we present several of our observations and summarize them as corollaries. These corollaries can be derived by following the definition of GAS.

Corollary 1 *The boolean or threshold-gates based access structure can be regarded as a proposition, which outputs **true** given an authorized set and outputs **false** otherwise.* $\square$

Corollary 2 *An access structure $\mathbb{A}$ can be regarded as a recursive data structure, meaning that any node x in $\mathbb{A}$ with its children nodes is also an access structure, denoted by $\mathbb{A}_x$. For example, there are 8 sub access structures in $\mathbb{A}$ of Figure 2.* $\square$

Corollary 3 *If $B \in \mathbb{A}$, there exists a path (i.e., a leaf node to the root node) in which each sub access structure is satisfied by B . We call it a satisfied path.* $\square$

3.3 Shamir Secret Sharing (SS)

Definition 4 (Shamir SS). *Shamir secret sharing (SS) scheme allows a dealer to split a secret into n shares for n shareholders. With only a threshold t ($\leq n$) shares, the secret can be successfully recovered, while fewer than t shares reveal nothing about the secret. Shamir SS is defined with following two algorithms.*

- $\{s_1, s_2, \dots, s_n\} \leftarrow \text{Share}(s, n, t)$: Inputs a secret $s \in \mathbb{Z}_q$, n and t , chooses a polynomial $f(x) = s + \sum_{i=1}^{t-1} a_i x^i$, where $a_i \xleftarrow{R} \mathbb{Z}_q$. Finally, the algorithm outputs n shares $f(i)$, $i \in [1, n]$.
- $s \leftarrow \text{Recon}(Q)$: Inputs any t shares $Q \subseteq \{s_1, s_2, \dots, s_n\}$, reconstructs the secret s via the Lagrange interpolation.

Definition 5 (Shamir SS on Group $\mathbb{G}$). Definition 4 has introduced the original Shamir secret sharing scheme, where a secret $s \in \mathbb{Z}_q$ is shared and recovered. Here we consider how to share secret $S \in \mathbb{G}$ when the dealer does not know the value s such that $S = g^s$. The Shamir secret sharing scheme on a group of order q is defined as below:

- $\{S_1, S_2, \dots, S_n\} \leftarrow \text{GrpShare}(S, n, t)$: Inputs a secret $S \in \mathbb{G}$, n and t , chooses a polynomial $f(x) = \sum_{i=1}^{t-1} a_i x^i$, where $a_i \xleftarrow{R} \mathbb{Z}_q$. Set $S_i = S \cdot g^{f(i)}$ for $i \in [1, n]$. The value $S_0 = S$ is only the interpolation target and is never output as a share.
- $S \leftarrow \text{GrpRecon}(Q)$: Inputs any t shares Q , reconstructs the secret $S \leftarrow S_0 = \prod_{i \in I} S_i^{\prod_{j \in I, j \neq i} \frac{-j}{i-j}}$, where I is the set containing indexes of the shares Q .

Shamir SS or Shamir SS on Group $\mathbb{G}$ should satisfy: **Correctness**: If the secret sharing process is correctly followed, any authorized set of participants can reconstruct the secret. **Secrecy**: No information about the secret is revealed to any unauthorized set of participants.

3.4 Linear Secret Sharing Scheme (LSSS)

Definition 6 (LSSS [33,22,20]). Let $\mathcal{P}$ be a set of shareholders. A linear secret sharing scheme over a ring $\mathbb{Z}_q$ for an access structure $\mathbb{A}$ is a tuple (M, ρ) , where $M \in \mathbb{Z}_q^{|\mathbb{A}| \times l}$ is a share-generating matrix with $|\mathbb{A}|$ rows and l columns¹ and ρ is a row-labeling function. The scheme consists of the following two algorithms:

- $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{LSSSShare}(s, \mathbb{A})$: To share a value $s \in \mathbb{Z}_q$ under access structure $\mathbb{A}$, randomly choose $v_2, \dots, v_{|\mathbb{A}|} \xleftarrow{R} \mathbb{Z}_q$ and set $\mathbf{v} = [s, v_2, \dots, v_{|\mathbb{A}|}]^\top$. Then, $[s_1, \dots, s_{|\mathbb{A}|}] \leftarrow M\mathbf{v}$ is the vector of shares, where $s_i \in \mathbb{Z}_q$ belongs to shareholder $\rho(i)$ for each $i \in [1, \dots, |\mathbb{A}|]$.
- $s \leftarrow \text{LSSSRecon}(\mathbb{A}, Q)$: Let $Q = \{(i, s_i)\}_{i \in I_Q}$ be a labelled set of available shares whose labels satisfy $\mathbb{A}$, where I_Q denotes the corresponding row indices of M . Let $M_Q \in \mathbb{Z}_q^{|I_Q| \times l}$ be the matrix formed by taking the subset of rows in M that are indexed by I_Q . Only if Q is an authorized set, there exists a vector $\mathbf{w}_Q \in \mathbb{Z}_q^{|I_Q|}$ such that $\mathbf{w}_Q^\top M_Q = [1, 0, \dots, 0]$. (Note that $\mathbf{w}_Q$ can be calculated using Gaussian elimination algorithm.) Further, the secret s can be reconstructed by $\sum_{i \in I_Q} \mathbf{w}_i s_i$.

¹ We can consider $\mathbb{A}$ as an access control tree, as illustrated in Figure 1 and Figure 2, where $|\mathbb{A}|$ denotes the number of leaf nodes—or equivalently, the number of secret shares. The value of l , however, depends on the specific parsing algorithm introduced in [20,56].

Similar to Shamir SS, LSSS should also satisfy **correctness** and **secrecy**.

3.5 Sigma Protocol and NIZK Proofs

Let $\mathcal{R}$ be a relation, and define the corresponding language as $L_{\mathcal{R}} = \{x \mid \exists w : (x, w) \in \mathcal{R}\}$. The values x and w are called the statement and the witness, respectively. A Sigma protocol (P_1, P_2, V) is a three-move public-coin protocol [29]: the prover sends a first message a , the verifier samples a challenge e , and the prover answers with a response z . The verifier accepts if $V(x, a, e, z) = 1$. Applying the Fiat-Shamir transform [30], the challenge is computed as a hash of the public statement and the first message.

Schnorr-Type NIZK Proofs: The basic Schnorr relation is $\mathcal{R}_{\text{Sch}} = \{((B, U), w) : U = B^w\}$. The prover samples $r \xleftarrow{R} \mathbb{Z}_q$, computes $A = B^r, c = H(B, U, A)$, and returns $\pi = (A, c, z)$, where $z = r - cw$. The verifier recomputes c and accepts only if $A = U^c B^z$. Indeed, for an honestly generated proof, $U^c B^z = (B^w)^c B^{r-cw} = B^r = A$.

DLEQ Proofs: The DLEQ proof is the Fiat-Shamir transformed Chaum-Pedersen protocol for the discrete-logarithm equality relation [57] $\mathcal{R}_{\text{DLEQ}} = \{((g, X, Y, Z), x) : X = g^x \wedge Z = Y^x\}$. A non-interactive DLEQ proof system consists of $\pi \leftarrow \text{DLEQ}((g, X, Y, Z), x)$ and $0/1 \leftarrow \text{DLEQVrf}(g, X, Y, Z, \pi)$, where the DLEQ algorithm uses a witness $x \in \mathbb{Z}_q$ satisfying $X = g^x$ and $Z = Y^x$. Concretely, the algorithm samples $r \xleftarrow{R} \mathbb{Z}_q$, computes $T_1 = g^r, T_2 = Y^r, e = H(g, X, Y, Z, T_1, T_2)$, and returns $\pi = (T_1, T_2, e, z)$, where $z = r - ex$. The DLEQVrf algorithm recomputes the challenge e and accepts only if $T_1 = X^e g^z$ and $T_2 = Z^e Y^z$. Completeness follows from $X^e g^z = (g^x)^e g^{r-ex} = g^r = T_1$ and $Z^e Y^z = (Y^x)^e Y^{r-ex} = Y^r = T_2$.

We rely on the standard properties of Fiat-Shamir transformed Sigma protocols in the random oracle model. **Completeness** states that honestly generated proofs for true statements are accepted. **Knowledge soundness** states that, except with negligible probability, any accepting proof implies the existence and efficient extractability of witnesses for the corresponding relation. **Zero-knowledge** states that proofs can be simulated without the witnesses and therefore reveal no information beyond the truth of the statements.

4 Generalized Secret Sharing (GSS)

We first define two types of GSS schemes, which differ based on the shared secret, $s \in \mathbb{Z}_q$ or $S \in \mathbb{G}$.

4.1 Definition

Definition 7 (GSS). A generalized secret sharing (GSS) scheme [16] allows a dealer to share a secret $s \in \mathbb{Z}_q$ using arbitrary monotone access structure. The

secret s can be recovered only given an authorized set. A GSS scheme is defined by the following two algorithms.

- $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{GSSShare}(s, \mathbb{A})$: Inputs a value $s \in \mathbb{Z}_q$ and an access structure $\mathbb{A}$, outputs the labelled secret shares $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\}$ in the canonical leaf order of $\mathbb{A}$, where $|\mathbb{A}|$ is the number of leaf nodes under access tree $\mathbb{A}$.
- $s \leftarrow \text{GSSRecon}(\mathbb{A}, Q)$: Inputs an access structure $\mathbb{A}$ and a labelled authorized share set $Q = \{(i, s_i)\}_{i \in I_Q}$, reconstructs the secret s .

Definition 8 (GSS on Group $\mathbb{G}$). Here we consider how to share secret $S \in \mathbb{G}$ and the dealer does not know the value s such that $S = g^s$. The GSS on group $\mathbb{G}$ of order q is defined as below:

- $\{S_1, S_2, \dots, S_{|\mathbb{A}|}\} \leftarrow \text{GrpGSSShare}(S, \mathbb{A})$: Inputs a value $S \in \mathbb{G}$ and an access structure $\mathbb{A}$, outputs the labelled group shares $\{S_1, S_2, \dots, S_{|\mathbb{A}|}\}$ in the same canonical leaf order as Definition 7.
- $S \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q)$: Inputs an access structure $\mathbb{A}$ and a labelled authorized group-share set $Q = \{(i, S_i)\}_{i \in I_Q}$, reconstructs the secret S .

Similar to Shamir SS, GSS and GSS on Group $\mathbb{G}$ should also satisfy **correctness** and **secrecy**.

4.2 Shamir SS-based GSS

Then, we initialize each type (Definition 7 and Definition 8) using two approaches, i.e., Shamir SS-based and LSSS-based. Technically, the Shamir SS-based primitive requires polynomial evaluation in the secret sharing and Lagrange interpolation in the secret recovery; The LSSS-based primitive needs to handle matrix multiplication both in secret sharing and recovery.

(1) **GSS based on Shamir SS**: Benaloh et al.[16] had introduced the construction, which can be obtained by replacing `GrpShare` (and `GrpRecon`) with `Share` (and `Recon`) in Figure 3. Note that the sharing phase is carried out by traversing the access control structure tree $\mathbb{A}$ from top to bottom, while the recovery process is performed by traversing the tree from bottom to top. We omit the security proofs of **correctness** and **secrecy**, which is proven by Benaloh et al.[16].

(2) **GSS on Group $\mathbb{G}$ based on Shamir SS**: Without loss of generalization, an access structure $\mathbb{A}$ is a set of shareholders who are constraint by the threshold gate (refer to Figure 2 as an example). The non-leaf nodes are threshold gate values and the leaf nodes are identifiers of shareholders. The GSS scheme attaches a share for each (non-leaf or leaf) node and it is described by Figure 3.

For any node in GSS scheme on $\mathbb{G}$, the **correctness** and **secrecy** can be derived directly from those of Shamir SS scheme on $\mathbb{G}$, which is given by Lemma 1. Thus, the **correctness** and **secrecy** of GSS on group $\mathbb{G}$ based on Shamir SS (Figure 3) are satisfied.

Functionality GSS on Group $\mathbb{G}$ using Shamir SS

$\{S_1, S_2, \dots, S_{|\mathbb{A}|}\} \leftarrow \text{GrpGSSShare}(S, \mathbb{A}) :$

set $F_R \leftarrow S$ for the root node R .

For non-leaf node x with n_x children nodes and threshold-gate t_x :

$\{S_{x_1}, S_{x_2}, \dots, S_{x_{n_x}}\} \leftarrow \text{GrpShare}(F_x, n_x, t_x)$

set $F_z \leftarrow S_{x_j}$, where z is the j th child of x

For the i th leaf node z in $\mathbb{A}$:

$S_i = S_{x_j}$, where z is the j th child of its parent node x

$(S) \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q) :$

Denote *path* as the satisfied path (cf. Corollary 3)

For each node x in *path* from leaf to root:

$F_x \leftarrow \text{GrpRecon}(Q_x)$, where $Q_x \subseteq \{F_{x_1}, \dots, F_{x_{n_x}}\}$ generated at node x and $|Q_x| \geq t_x$. (**Remark:** if x is a leaf node, $F_x = Q_x$.)

$S \leftarrow F_R$, where R is the root node.

Fig. 3: GSS scheme on Group $\mathbb{G}$ based on Shamir SS

Lemma 1. *Shamir SS on group $\mathbb{G}$ (Definition 5) satisfies **correctness** and **secrecy**.*

Proof. **Correctness:** The shares are $S_i = S \cdot g^{f(i)}$ for $i \in [1, n]$, and the interpolation target is $S_0 = S$. Following the correctness of Shamir SS scheme, inputs a set of any t shares Q , we can reconstruct the secret $\prod_{i \in I} S_i^{\prod_{j \in I, j \neq i} \frac{-j}{i-j}} = S \cdot g^{f(0)} = S$, where I is the set containing indexes of the shares Q . **Secrecy:** If less than t shares can reconstruct the secret, then we have $\sum_{i \in I} f(i) \prod_{j \in I, j \neq i} \frac{-j}{i-j} = f(0)$, $|I| < t$, which violates the security properties of Shamir SS scheme. $\square$

4.3 LSSS-based GSS

(1) **GSS based on LSSS:** It is obvious that LSSS and GSS schemes have the same algorithms, including inputs and outputs. Figure 4 demonstrate that LSSS is an instance of GSS. Therefore, the **correctness** and **secrecy** are inherently guaranteed.

Functionality GSS using LSSS

$\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{GSSShare}(s, \mathbb{A}) :$

$\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{LSSSShare}(s, \mathbb{A})$

$(s) \leftarrow \text{GSSRecon}(\mathbb{A}, Q) :$

$s \leftarrow \text{LSSSRecon}(\mathbb{A}, Q)$

Fig. 4: GSS based on LSSS

(2) **GSS on Group $\mathbb{G}$ using LSSS:** We first depict how to construct LSSS on group $\mathbb{G}$ by Figure 5. The construction resembles the LSSSShare and

LSSSRecon algorithms of the LSSS scheme, as defined in Definition 6. We prove that it satisfies the **correctness** and **secrecy** by Lemma 2.

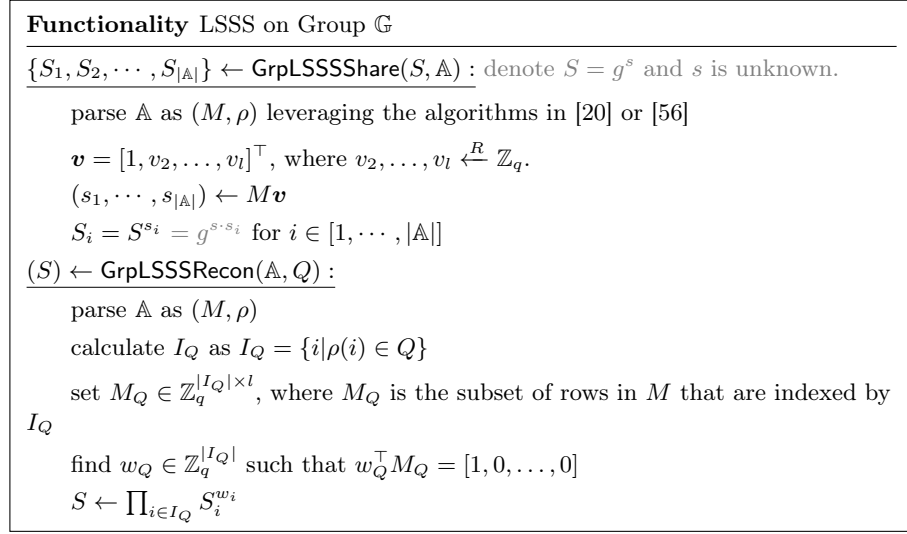


Fig. 5: LSSS on Group $\mathbb{G}$

Lemma 2. *LSSS on group $\mathbb{G}$ (Figure 5) satisfies **correctness** and **secrecy**.*

Proof. Correctness: The shares are $S_i = S^{s_i} = g^{s \cdot s_i}$ for $i \in [1, \dots, |\mathbb{A}|]$. Q is an authorized set with $I_Q = \{i | \rho(i) \in Q\}$ and $M_Q \in \mathbb{Z}_q^{|I_Q| \times l}$, which means there exists $w_Q \in \mathbb{Z}_q^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \dots, 0]$. Following the correctness of LSSS scheme, we have $\sum_{i \in I_Q} w_i \cdot s_i = 1$, thus $S = \prod_{i \in I_Q} S_i^{w_i} = \prod_{i \in I_Q} g^{s \cdot s_i \cdot w_i} = (g^s)^{\sum_{i \in I_Q} w_i \cdot s_i} = g^s$. **Secrecy:** If unauthorized set Q can reconstruct $g^s = \prod_{i \in I_Q} S_i^{w_i}$, then it can find $w_Q \in \mathbb{Z}_q^{|I_Q|}$ such that $w_Q^\top M_Q = [1, 0, \dots, 0]$, which violates the security properties of LSSS scheme, as only authorized sets can find such w_Q . $\square$

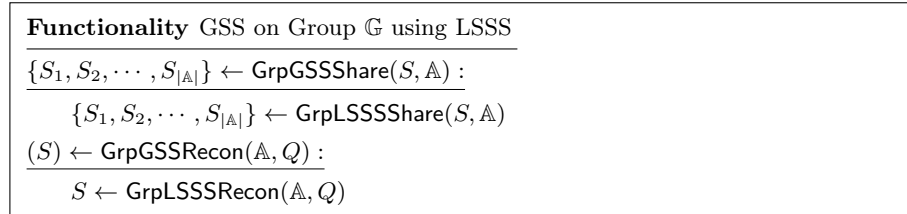


Fig. 6: GSS on Group $\mathbb{G}$ based on LSSS

Then, we apply the construction of Figure 5 to construct GSS on group $\mathbb{G}$, as Figure 6 presents. It can be observed that the algorithm GrpGSSShare

(w.r.t. GrpGSSRecon directly executes GrpLSSSShare (w.r.t. GrpLSSSRecon). Apparently, LSSS on group $\mathbb{G}$ can be regarded as an instance of GSS on group $\mathbb{G}$. Thus, **correctness** and **secrecy** are inherently guaranteed by Lemma 2.

4.4 GSS Testing

The reconstruction algorithm GSSRecon only needs an authorized subset of shares. This is sufficient for recovering the secret, but it is not sufficient for public verification in PVGSS: the verifier must check that the entire vector appearing in the proof response is one globally consistent sharing of a claimed response secret. Otherwise, a malicious dealer could make some authorized subsets reconstruct correctly while other leaves are inconsistent with the declared access structure. We capture this global consistency check as GSS testing.

Definition 9 (GSS Testing). *Let $\mathbb{A}$ be an access structure with $|\mathbb{A}|$ leaves in canonical order. A GSS testing algorithm*

$$b \leftarrow \text{GSSTest}(\mathbb{A}, \mathbf{x}, x)$$

takes a candidate share vector $\mathbf{x} = (x_1, \dots, x_{|\mathbb{A}|}) \in \mathbb{Z}_q^{|\mathbb{A}|}$ and a candidate secret $x \in \mathbb{Z}_q$, and outputs $b \in \{0, 1\}$. It is complete if $\text{GSSTest}(\mathbb{A}, \mathbf{x}, x) = 1$ for every honestly generated vector $\mathbf{x} \leftarrow \text{GSSShare}(x, \mathbb{A})$. It is sound with error ϵ if, except with probability at most ϵ , acceptance implies that $\mathbf{x}$ is a valid output of $\text{GSSShare}(x, \mathbb{A})$.

We use the following two testing procedures for threshold-gate access trees, using the notation of Definitions 1 and 2. These tests apply not only to the recursive Shamir realization of GSS, but also to the LSSS realization used in this work. Specifically, our LSSS matrix is obtained from a threshold-gate access tree by the conversion of Liu et al. [56]. In that conversion, expanding a t -of- m gate appends a local Vandermonde block over $\mathbb{Z}_q$, so the values assigned to the children of each internal gate are still evaluations of a degree- $< t$ polynomial whose constant term is the value assigned to the parent. Hence the resulting LSSS share vector preserves the same local Reed-Solomon structure as the tree representation, and GSS testing can be performed on the canonical leaf order induced by the tree.

For an internal node v , let t_v be its threshold, let m_v be the number of its children, and let $\alpha_{v,1}, \dots, \alpha_{v,m_v} \in \mathbb{Z}_q^*$ be their distinct evaluation points. Let r denote the root of $\mathbb{A}$, and let $Q = \{(i, x_i)\}_{i \in [|\mathbb{A}|]}$ denote the candidate leaf shares represented by $\mathbf{x}$.

① *Exact interpolation test.* Sample a minimal authorized subset $Q^* \subseteq Q$. Equivalently, for each internal node v , choose a set $I_v \subseteq [m_v]$ of size t_v that satisfies v , and recurse on the selected children until the selected leaves are exactly Q^* . The test assigns the claimed value $z_r := x$ to the root. For every internal node v , after the values $\{z_{v,j}\}_{j \in I_v}$ of the selected children have been obtained recursively, define the unique polynomial

$$f_v(X) = \sum_{j \in I_v} z_{v,j} \ell_j^{I_v}(X) \quad \text{with} \quad \deg(f_v) < t_v.$$

Here the basis polynomials $\ell_j^{I_v}$ are defined with respect to the local evaluation points $\{\alpha_{v,j}\}_{j \in I_v}$. The local root consistency condition is $f_v(0) = z_v$. For every unselected child $j \notin I_v$, set $z_{v,j} := f_v(\alpha_{v,j})$ and verify the corresponding child subtree using $z_{v,j}$ as its claimed value; for a leaf child, this verification is the equality check $z_{v,j} = x_i$ against the corresponding entry of Q . The test accepts if and only if all local root-consistency checks and all leaf comparisons accept. Acceptance therefore determines, for every internal node v , a degree- $< t_v$ polynomial whose evaluations agree with all candidate descendants. Hence the accepted vector is a valid recursive Shamir sharing of x . The random choice of Q^* only fixes an interpolation basis and does not introduce soundness error.

② *Dual-code test.* For each internal node v , form the extended evaluation-point vector

$$\bar{\alpha}_v = (0, \alpha_{v,1}, \dots, \alpha_{v,m_v})$$

and the corresponding local Reed-Solomon code

$$\mathcal{C}_v = \text{RS}_{\bar{\alpha}_v}(m_v + 1, t_v).$$

A locally consistent assignment at v is precisely a vector $(z_v, z_{v,1}, \dots, z_{v,m_v}) \in \mathcal{C}_v$, where z_v is the value assigned to v and $z_{v,j}$ is the value assigned to its j -th child. The verifier reconstructs the internal-node values from the leaves in a bottom-up manner and checks the root condition $z_r = x$. Simultaneously, for each v , it samples a random dual constraint $\mathbf{h}_v \in \mathcal{C}_v^\perp$, or equivalently a random linear combination of a basis of $\mathcal{C}_v^\perp$, and checks

$$\langle \mathbf{h}_v, (z_v, z_{v,1}, \dots, z_{v,m_v}) \rangle = 0.$$

Equivalently, all local dual constraints may be aggregated into one random global parity check. If the candidate assignment is not a valid recursive Shamir sharing of x , then at least one local syndrome is nonzero or the root condition fails. A random dual-code check misses a nonzero syndrome with probability at most $1/q$ [11]; therefore the soundness error is bounded by $1/q$.

Lemma 3 (Soundness of GSS Testing). *For recursive Shamir GSS over $\mathbb{Z}_q$, and for the Liu et al. tree-to-LSSS realization used in this paper, the exact interpolation test is complete and sound with error 0. The dual-code test is complete and sound with error at most $1/q$.*

Proof. Completeness of both procedures follows from the local correctness of Shamir sharing at every threshold gate. In an honestly generated recursive sharing, the values assigned to the children of each internal t -of- m gate are evaluations of a polynomial of degree less than t , whose constant term is the value assigned to that gate. Therefore, Lagrange interpolation reconstructs the correct local polynomial, all recomputed child values agree with the candidate vector, all dual-code parity checks vanish, and the root value is the shared secret x .

For soundness of the exact interpolation test, suppose the test accepts. The interpolation phase fixes, for every internal gate, a polynomial of degree less than the gate threshold. The subsequent comparison phase verifies that every

unselected child value agrees with the corresponding evaluation of this polynomial, while the selected child values agree by construction. By induction from the leaves to the root, every subtree is therefore a valid recursive Shamir sharing of the value reconstructed at its root. Since the final root check requires this value to equal x , the full vector $\mathbf{x}$ is a valid sharing of x . Thus the exact test has soundness error 0.

For the dual-code test, any invalid candidate vector violates at least one local Reed-Solomon consistency condition or reconstructs a root value different from x . The violated local consistency condition corresponds to a nonzero syndrome with respect to the relevant dual Reed-Solomon code. A random linear combination of the induced dual constraints evaluates to zero on this nonzero syndrome with probability at most $1/q$ over the verifier's random choice. Thus an invalid vector is accepted with probability at most $1/q$. $\square$

5 Publicly Verifiable Generalized Secret Sharing

5.1 Definition

Definition 10 (PVGSS). *A publicly verifiable generalized secret sharing (PVGSS) scheme allows a dealer to split a secret using an arbitrary monotone access structure in a publicly verifiable manner. Shares are encrypted and can be delivered over a public channel. The correctness of encrypted shares and decrypted reconstruction shares can be verified by any external party. The ordered public-key list is always aligned with the canonical leaf order of the access structure. The PVGSS scheme includes the following algorithms.*

- $(\mathbf{sk}_i, \mathbf{pk}_i) \leftarrow \text{PVGSSSetup}(\lambda)$: Inputs security parameter λ , outputs a key pair $(\mathbf{sk}_i, \mathbf{pk}_i)$ for shareholder P_i .
- $(\{C_i\}, \text{prf}_s) \leftarrow \text{PVGSSShare}(s, \mathbb{A}, \{\mathbf{pk}_i\})$: Inputs a secret $s \in \mathbb{Z}_q$, an access structure $\mathbb{A}$, and an ordered list of public keys $\{\mathbf{pk}_i\}_{i \in [\mathbb{A}]}$, outputs encrypted shares $\{C_i\}$ and the corresponding NIZK proof prf_s .
- $(0/1) \leftarrow \text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}, \{\mathbf{pk}_i\})$: Inputs encrypted shares $\{C_i\}$, proof prf_s , access structure $\mathbb{A}$, and ordered public keys, outputs 1 if the shares are correctly encrypted. Otherwise, outputs 0.
- $(S_i, \text{prf}_{S_i}) \leftarrow \text{PVGSSPreRecon}(C_i, \mathbf{sk}_i)$: Inputs ciphertext C_i and the corresponding secret key $\mathbf{sk}_i$, outputs a decrypted share S_i and the attached NIZK proof prf_{S_i} .
- $(0/1) \leftarrow \text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, \mathbf{pk}_i)$: Inputs the encrypted share C_i , the decrypted share S_i , the NIZK proof prf_{S_i} , and public key $\mathbf{pk}_i$, outputs 1 if S_i is a correct decryption of C_i .
- $(S) \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q)$: Inputs the access structure $\mathbb{A}$ and a labelled authorized set of decrypted shares $Q = \{(i, S_i)\}_{i \in I_Q}$, outputs the recovered secret $S(= g^s)$.

PVGSS scheme is required to achieve below security properties, which are formally defined in Appendix F.

- **Correctness:** The scheme ensures that if the secret sharing process is correctly followed, any authorized subset of participants can reconstruct the secret S .
- **Verifiability:** Anyone can verify that the encrypted shares provided by the dealer are correct without knowing the actual secret. Besides, users can also check correctness of the decrypted shares in the reconstruction phase.
- **IND1-secrecy:** The secret remains hidden from unauthorized parties prior to the reconstruction phase.

5.2 PVGSS Construction Based on GSS

Usually, PVSS schemes are constructed based on SS. Informally speaking, PVSS combines SS and public key encryption with verifiability. Intuitively, PVGSS can be constructed similarly by replacing SS with GSS. We leverage the original GSS scheme in the sharing phase and employ GSS on group $\mathbb{G}$ in the reconstruction phase. Figure 7 illustrate our mindset of constructions vividly. Figure 8 demonstrates the general PVGSS construction by taking ad-

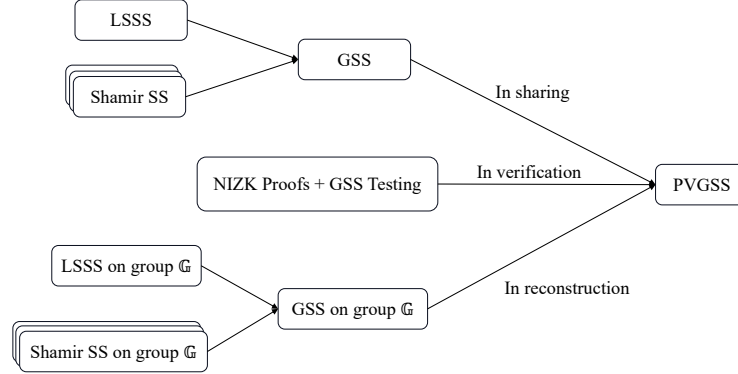


Fig. 7: High-level overview of our PVGSS constructions

vantage of GSS scheme and NIZK proofs. The sharing proofs are Schnorr-type NIZK proofs obtained through the Fiat-Shamir transform [30]. The initiation phase comprises the PVGSSSetup algorithm. The sharing phase involves the PVGSSShare and PVGSSVerify algorithms. The reconstruction phase includes PVGSSPreRecon , PVGSSKeyVrf , and PVGSSRecon algorithms.

In the PVGSSSetup algorithm, each shareholder P_i generates its key pair $(\text{sk}_i, \text{pk}_i)$, where $\text{pk}_i = g^{\text{sk}_i}$ and sk_i is randomly chosen from $\mathbb{Z}_q^*$.

In the PVGSSShare algorithm, the dealer first invokes GSSShare algorithm for secret $s \in \mathbb{Z}_q$ to generate secret shares based on the access structure $\mathbb{A}$. Note that a PVGSS is not merely a combination of multiple PVSS instances, since it outputs encrypted shares only for the leaf nodes. As previously introduced, $|\mathbb{A}|$ denotes the number of leaf nodes in the access structure $\mathbb{A}$. Then, it hides the $|\mathbb{A}|$ shares directly using the targeted shareholder's public key in the same leaf order, i.e., $\text{pk}_i^{\text{sk}_i}$. Next, it generates Schnorr-type Fiat-Shamir NIZK proofs.

Specifically, $\{C'_i\}$, c , and $\{\hat{s}_i\}$ are the commitment, the challenge value, and the response values, respectively. The challenge binds the ordered public keys and the access structure. Therefore, the NIZK proof prf_s is $(\{C'_i\}, c, \hat{s}, \{\hat{s}_i\})$. To this end, the dealer has successfully generated encrypted secret shares $\{C_i\}$ and the corresponding NIZK proof prf_s which can be adopted to prove the dealer's honesty. Different from the LDEI protocol in Albatross [35], which directly outputs the polynomial $\hat{p}(X)$ as part of the NIZK response values, PVGSS transmits $|\mathbb{A}|$ polynomial evaluation points $\{\hat{s}_i\}$ of $\hat{p}(X)$.

In the PVGSSVerify algorithm, any (external) party can check whether the dealer has honestly performed the sharing phase. Due to the presence of $|\mathbb{A}|$ independent shares $\{s_i\}$, the verifier checks group membership, recomputes the Fiat-Shamir challenge, and then verifies two algebraic conditions. The first equation (executed $|\mathbb{A}|$ times) is a Schnorr-type proof of knowledge for each encrypted share s_i . The second check invokes GSSTest directly to verify that all response values $\{\hat{s}_i\}$ are valid GSS shares of the response secret $\hat{s}$. It binds the shares embedded in $\{C_i\}$ to the secret s under the access structure $\mathbb{A}$. Refer to Theorem 1 for details about the correctness.

In the PVGSSPreRecon algorithm, the shareholder, holding sk_i , decrypts C_i with its secret key sk_i and obtains $S_i = g^{s_i}$. Additionally, the shareholder employs the discrete logarithm equality (DLEQ) algorithm [57] to generate a non-interactive zero-knowledge (NIZK) proof prf_{S_i} , demonstrating that $\log_g \text{pk}_i = \log_{S_i} C_i = \text{sk}_i$, i.e., $C_i = S_i^{\text{sk}_i}$.

The PVGSSKeyVrf algorithm directly invokes the corresponding DLEQ verification algorithm, DLEQVrf, to verify the correctness of S_i provided by the shareholder. Note that no information about sk_i is leaked during this process.

Given enough authorized decrypted shares, i.e., an authorized set Q , the PVGSSRecon algorithm can reconstruct a secret S using GrpGSSRecon algorithm. Note that GrpGSSRecon, rather than GSSRecon, is used to recover the secret, since in our PVGSS scheme the secret S resides in $\mathbb{G}$, consistent with many prior PVSS constructions [11,35].

5.3 Complexity Analysis

Computational Complexity: We first count the dominant group operations in $\mathbb{G}$, and then account for the public finite-field computations that affect the concrete running time. Matrix operations in the LSSS-based construction, polynomial evaluation and Lagrange interpolation in the Shamir SS-based construction, Exact/Dual GSS testing, and LSSS reconstruction-weight computation do not change the group-operation asymptotics. However, they are not negligible in the implementation, and they explain part of the curve separation observed in the experiments.

In the sharing phase, the PVGSSShare algorithm costs $2|\mathbb{A}|$ group exponentiations, where each C_i and C'_i requires an exponentiation. The PVGSSVerify algorithm requires $2|\mathbb{A}|$ exponentiations to verify the NIZK equation for each encrypted share, plus one public GSS testing procedure over the responses. The

Functionality PVGSS Based on GSS

$(\text{sk}_i, \text{pk}_i) \leftarrow \text{PVGSSSetup}(\lambda) :$
 $\text{sk}_i \xleftarrow{R} \mathbb{Z}_q^*$
 $\text{pk}_i \leftarrow g^{\text{sk}_i}$

$(\{C_i\}, \text{prf}_s) \leftarrow \text{PVGSSShare}(s, \mathbb{A}, \{\text{pk}_i\}) :$
 reject if any pk_i is invalid or the list order is ambiguous
 $\{s_1, s_2, \dots, s_{|\mathbb{A}|}\} \leftarrow \text{GSSShare}(s, \mathbb{A})$
 $C_i = \text{pk}_i^{s_i} \ \forall i \in [1, \dots, |\mathbb{A}|]$
 $s' \xleftarrow{R} \mathbb{Z}_q$
 $(s'_1, \dots, s'_{|\mathbb{A}|}) = \text{GSSShare}(s', \mathbb{A})$
 $C'_i = \text{pk}_i^{s'_i} \ \forall i \in [1, \dots, |\mathbb{A}|]$
 $c \leftarrow H(\mathbb{A}, \{\text{pk}_i\}, \{C_i\}, \{C'_i\})$
 $\hat{s} \leftarrow s' - c \cdot s$
 $\hat{s}_i \leftarrow s'_i - c \cdot s_i \ \forall i \in [1, \dots, |\mathbb{A}|]$
 $\text{prf}_s \leftarrow (\{C'_i\}, c, \hat{s}, \{\hat{s}_i\})$

$(0/1) \leftarrow \text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}, \{\text{pk}_i\}) :$
 reject if any pk_i , C_i , or C'_i is invalid
 $c \stackrel{?}{=} H(\mathbb{A}, \{\text{pk}_i\}, \{C_i\}, \{C'_i\})$
 $C'_i \stackrel{?}{=} C_i^c \cdot \text{pk}_i^{\hat{s}_i} \ \forall i \in [1, \dots, |\mathbb{A}|]$
 $1 \stackrel{?}{=} \text{GSSTest}(\mathbb{A}, \{\hat{s}_i\}_{i \in [1, \dots, |\mathbb{A}|]}, \hat{s})$
 return 1 iff all checks pass

$(S_i, \text{prf}_{S_i}) \leftarrow \text{PVGSSPreRecon}(C_i, \text{sk}_i) :$
 reject if C_i is invalid
 $S_i = C_i^{1/\text{sk}_i} = g^{s_i}$
 $\text{prf}_{S_i} = \text{DLEQ}((g, \text{pk}_i, S_i, C_i), \text{sk}_i)$

$(0/1) \leftarrow \text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, \text{pk}_i) :$
 reject if pk_i , C_i , or S_i is invalid
 $\text{true} \stackrel{?}{=} \text{DLEQVrf}(g, \text{pk}_i, S_i, C_i, \text{prf}_{S_i})$

$(S) \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q) :$
 $S \leftarrow \text{GrpGSSRecon}(\mathbb{A}, Q)$ where $S = g^s$ and $Q = \{(i, S_i)\}_{i \in I_Q}$ is a labelled authorized set of $\mathbb{A}$.

Fig. 8: PVGSS scheme based on GSS and NIZK proofs

Exact interpolation test and the Dual-code test are both public finite-field checks over the access structure; they preserve linear group-operation complexity but lead to different concrete verification costs. Therefore, the distribution complexity is $O(|\mathbb{A}|)$ in group operations, with implementation-level constants affected by

the chosen GSS testing routine. In the reconstruction phase, the PVGSSPreRecon algorithm costs a constant number of exponentiations for each shareholder, including the calculation of prf_{S_i} from DLEQ. The PVGSSKeyVrf algorithm requires 4 exponentiations as required by DLEQVrf. The online PVGSSRecon algorithm takes $|Q|$ exponentiations, where Q denotes the authorized set for the access structure $\mathbb{A}$. For LSSS-based reconstruction, the public reconstruction weights may be computed from the access matrix and authorized set; for fixed DEX access structures and authorized sets these weights can be precomputed, so the online reconstruction remains a group linear combination. For Shamir SS-based reconstruction, Lagrange interpolation on the selected path contributes to the measured running time. Thus, the online reconstruction complexity is $O(|Q|)$ in group operations, while the concrete cost depends on whether these public coefficients are prepared in advance.

Communicational Complexity: In the PVGSS sharing phase, the dealer outputs $(\{C_i\}, \text{prf}_s)$, which consists of $2|\mathbb{A}|$ $\mathbb{G}$ elements and $(|\mathbb{A}| + 2) \mathbb{Z}_q$ elements. During the reconstruction phase, each user needs to collect $(3\mathbb{G} + 2\mathbb{Z}_q)|Q|$ elements, as each correct prf_{S_i} contains 2 elements on $\mathbb{G}$ and 2 elements on $\mathbb{Z}_q$. Therefore, the communication complexity is $O(|\mathbb{A}|)$ during the sharing phase and $O(|Q|)$ during the reconstruction phase, respectively.

5.4 Security Analysis

Theorem 1 (Correctness). *The PVGSS construction satisfies correctness defined by Definition 11.*

Proof. Fix an arbitrary security parameter λ , access structure $\mathbb{A}$, secret $s \in \mathbb{Z}_q$, and ordered valid public-key list $\{\text{pk}_i\}_{i \in [|\mathbb{A}|]}$, where $\text{pk}_i = g^{\text{sk}_i}$ and $\text{sk}_i \in \mathbb{Z}_q^*$. Let

$$(\{C_i\}, \text{prf}_s) \leftarrow \text{PVGSSShare}(s, \mathbb{A}, \{\text{pk}_i\})$$

be an honest sharing. By the definition of PVGSSShare, the dealer first samples a valid GSS sharing

$$(s_1, \dots, s_{|\mathbb{A}|}) \leftarrow \text{GSSShare}(s, \mathbb{A}), \quad C_i = \text{pk}_i^{s_i},$$

and publishes $C_i = \text{pk}_i^{s_i}$ for every leaf i . It also samples $s' \in \mathbb{Z}_q$, computes

$$(s'_1, \dots, s'_{|\mathbb{A}|}) \leftarrow \text{GSSShare}(s', \mathbb{A}), \quad C'_i = \text{pk}_i^{s'_i},$$

sets

$$c = H(\mathbb{A}, \{\text{pk}_i\}, \{C_i\}, \{C'_i\}), \quad \hat{s} = s' - cs, \quad \hat{s}_i = s'_i - cs_i.$$

We first show that PVGSSVerify accepts. Since all values are generated honestly, the verifier recomputes the same Fiat-Shamir challenge c . Moreover, for every i ,

$$C_i^c \text{pk}_i^{\hat{s}_i} = (\text{pk}_i^{s_i})^c \text{pk}_i^{s'_i - cs_i} = \text{pk}_i^{s'_i} = C'_i.$$

Thus all Schnorr-type verification equations hold. It remains to check the GSS-consistency equation. Because the underlying GSS sharing algorithm is linear,

$$(\hat{s}_1, \dots, \hat{s}_{|\mathbb{A}|}) = (s'_1, \dots, s'_{|\mathbb{A}|}) - c(s_1, \dots, s_{|\mathbb{A}|})$$

is a valid GSS sharing of $\hat{s} = s' - cs$ under the same access structure $\mathbb{A}$. By completeness of GSSTest ,

$$\text{GSSTest}(\mathbb{A}, \{\hat{s}_i\}_{i \in [|\mathbb{A}|]}, \hat{s}) = 1.$$

Therefore,

$$\text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}, \{\text{pk}_i\}) = 1.$$

Next, consider any labelled authorized set $Q = \{(i, S_i)\}_{i \in I_Q}$ of $\mathbb{A}$. For every $i \in I_Q$, honest execution of PVGSSPreRecon gives

$$S_i = C_i^{1/\text{sk}_i} = (\text{pk}_i^{\text{sk}_i})^{1/\text{sk}_i} = (g^{\text{sk}_i s_i})^{1/\text{sk}_i} = g^{s_i}.$$

The proof prf_{S_i} generated by PVGSSPreRecon is an honest DLEQ proof for the statement

$$\log_g \text{pk}_i = \log_{S_i} C_i = \text{sk}_i,$$

since $C_i = S_i^{\text{sk}_i}$. By completeness of the DLEQ proof system,

$$\text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, \text{pk}_i) = 1 \quad \text{for all } i \in I_Q.$$

Finally, since Q is authorized and the values supplied to reconstruction are exactly the group shares $S_i = g^{s_i}$ corresponding to a valid GSS sharing of s , correctness of the underlying GSS-on-group reconstruction algorithm gives

$$S' \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q) = \text{GrpGSSRecon}(\mathbb{A}, Q) = g^s.$$

This correctness of GrpGSSRecon follows from the corresponding group-GSS instantiations proved in Sections 4.2 and 4.3.

Combining the three conclusions, for every honestly generated execution and every authorized labelled set Q , all verification algorithms accept and the reconstructed value equals g^s . Hence the PVGSS construction satisfies correctness by Definition 11. $\square$

Theorem 2 (Verifiability of Encrypted Shares). *If the Schnorr-type Fiat-Shamir proof used in PVGSSShare is knowledge-sound in the random oracle model and the instantiated GSS testing procedure is sound with error ϵ_{test} , then PVGSS satisfies verifiability of encrypted shares, as defined by Definition 12, up to the corresponding testing error. In particular, for recursive Shamir GSS, Lemma 3 gives $\epsilon_{\text{test}} = 0$ for the exact interpolation test and $\epsilon_{\text{test}} \leq 1/q$ for the dual-code test.*

Proof. Let $\text{Exp}_{\mathcal{A}}^{\text{venc}}$ denote the experiment in Definition 12. The experiment outputs 1 only if $\mathcal{A}$ produces $(\{C_i\}, \text{prf}_s)$ such that PVGSSVerify accepts, but there is no secret s and no valid GSS sharing $\{s_i\}$ of s satisfying $C_i = \text{pk}_i^{s_i}$ for all i .

Consider an accepting proof $\text{prf}_s = (\{C'_i\}, c, \hat{s}, \{\hat{s}_i\})$, for the statement $(\mathbb{A}, \{\text{pk}_i\}, \{C_i\})$. By knowledge soundness of the Fiat-Shamir Schnorr proof in the ROM, except with probability

$$\text{Adv}_{\text{FS}}^{\text{ks}}(\mathcal{B}_{\text{FS}}) + \text{negl}(\lambda)$$

for a PPT extractor adversary $\mathcal{B}_{\text{FS}}$, the extractor obtains a second accepting transcript with the same Sigma-protocol commitment and a distinct challenge:

$$(\{C'_i\}, \tilde{c}, \tilde{s}, \{\tilde{s}_i\}) \quad \text{where } c \neq \tilde{c}.$$

Both transcripts pass the algebraic verification equation, and hence, for every leaf i ,

$$C'_i = C_i^c \text{pk}_i^{\hat{s}_i} = C_i^{\tilde{c}} \text{pk}_i^{\tilde{s}_i}.$$

Since all public keys are validated as non-identity elements of the prime-order group and $c - \tilde{c} \neq 0$, define

$$s_i = \frac{\tilde{s}_i - \hat{s}_i}{c - \tilde{c}} \quad \text{and} \quad s = \frac{\tilde{s} - \hat{s}}{c - \tilde{c}}.$$

The preceding equation immediately gives $C_i = \text{pk}_i^{s_i}$ for every i .

It remains to show that the extracted exponents form a valid GSS sharing of s . Since both transcripts are accepting, they also satisfy

$$\text{GSSTest}(\mathbb{A}, \{\hat{s}_i\}, \hat{s}) = 1 \quad \text{and} \quad \text{GSSTest}(\mathbb{A}, \{\tilde{s}_i\}, \tilde{s}) = 1.$$

By soundness of GSS testing, except with probability at most $2\epsilon_{\text{test}}$, the vectors $\{\hat{s}_i\}$ and $\{\tilde{s}_i\}$ are valid GSS sharings of $\hat{s}$ and $\tilde{s}$, respectively. The underlying GSS sharing relation is linear, so the coordinate-wise linear combination

$$\left\{ \frac{\tilde{s}_i - \hat{s}_i}{c - \tilde{c}} \right\}_{i \in [\mathbb{A}]}$$

is a valid GSS sharing of $(\tilde{s} - \hat{s})/(c - \tilde{c}) = s$. Therefore the accepted ciphertext vector encrypts valid GSS shares of the secret s , which contradicts the winning condition in Definition 12.

Consequently, any successful adversary must either break knowledge soundness of the Fiat-Shamir proof or cause an invalid vector to pass one of the two GSS tests. Thus

$$\Pr[\text{Exp}_{\mathcal{A}}^{\text{venc}} = 1] \leq \text{Adv}_{\text{FS}}^{\text{ks}}(\mathcal{B}_{\text{FS}}) + 2\epsilon_{\text{test}} + \text{negl}(\lambda).$$

For recursive Shamir GSS, Lemma 3 gives $\epsilon_{\text{test}} = 0$ for the exact interpolation test and $\epsilon_{\text{test}} \leq 1/q$ for the dual-code test, yielding the claimed bound. $\square$

Theorem 3 (Verifiability of Share Decryption). *If the DLEQ proof system used in PVGSSPreRecon satisfies the standard soundness property, then PVGSS satisfies verifiability of share decryption as defined by Definition 13.*

Proof. Let $\text{Exp}_{\mathcal{A}}^{\text{vdec}}$ be the experiment in Definition 13; the experiment outputs 1 when $\mathcal{A}$ produces $(\text{pk}, C, S, \text{prf}_S)$ such that $\text{PVGSSKeyVrf}(C, S, \text{prf}_S, \text{pk}) = 1$, but there is no $x \in \mathbb{Z}_q^*$ satisfying $\text{pk} = g^x$ and $C = S^x$.

Since PVGSSKeyVrf first validates pk, C, S and then invokes $\text{DLEQVrf}(g, \text{pk}, S, C, \text{prf}_S)$, an accepting adversarial output is an accepting DLEQ proof for the statement (g, pk, S, C) . By DLEQ soundness, except with negligible probability, there exists $x \in \mathbb{Z}_q$ such that $\text{pk} = g^x$ and $C = S^x$. Registered public keys are validated as non-identity group elements, so this witness is nonzero and hence lies in $\mathbb{Z}_q^*$. Therefore the winning condition of $\text{Exp}_{\mathcal{A}}^{\text{vdec}}$ can occur only with negligible probability:

$$\Pr[\text{Exp}_{\mathcal{A}}^{\text{vdec}} = 1] \leq \text{negl}(\lambda).$$

In particular, when $C = \text{pk}_i^{s_i}$ is a PVGSS ciphertext for registered $\text{pk}_i = g^{\text{sk}_i}$, the accepted DLEQ relation gives $C = S^{\text{sk}_i}$, hence $S = C^{1/\text{sk}_i} = g^{s_i}$. Therefore an invalid decrypted share is rejected except with negligible DLEQ soundness error. $\square$

Theorem 4 (IND1-secrecy). *The PVGSS constructions satisfy IND1-secrecy, defined by game Definition 14, assuming (i) zero-knowledge of the Fiat-Shamir proofs in the random oracle model, (ii) DDH in $\mathbb{G}$, and (iii) privacy of the underlying GSS-on-group scheme against unauthorized labelled sets.*

Proof. Let $\mathcal{A}$ be a PPT adversary in the IND1 game. Let $m = |\mathbb{A}^*|$, and let $U \subseteq [m]$ be the unauthorized set chosen by $\mathcal{A}$. We write E_j for the event that $\mathcal{A}$ outputs $b' = b$ in Game G_j .

Game G_0 . This is the real IND1 experiment of Definition 14. The challenger samples b , runs

$$(\{C_i^*\}, \text{prf}_{x_b}) \leftarrow \text{PVGSSShare}(x_b, \mathbb{A}^*, \{\text{pk}_i\}),$$

and answers every allowed decryption query $i \in U$ with $(S_i^*, \text{prf}_{S_i}^*) \leftarrow \text{PVGSSPreRecon}(C_i^*, \text{sk}_i)$.

Thus $\text{Adv}_{\text{PVGSS}}^{\text{ind1}}(\mathcal{A}) = |\Pr[E_0] - 1/2|$.

Game G_1 : simulate sharing proofs. This game is identical to G_0 , except that the NIZK proof prf_{x_b} generated in the sharing phase is replaced by a simulated proof for the same public statement. Since the proof is zero-knowledge in the random oracle model,

$$|\Pr[E_0] - \Pr[E_1]| \leq \text{Adv}_{\text{FS}}^{\text{zk}}(\mathcal{B}_{\text{zk}}) + \text{negl}(\lambda).$$

The DLEQ proofs returned for allowed decryption queries are still honestly generated, since these queries correspond to real decrypted group shares.

Game G_2 : randomize unopened ciphertexts. Starting from G_1 , replace each unopened ciphertext $C_i^* = \text{pk}_i^{s_i}$ for $i \notin U$ by an independently uniform group element $R_i \in \mathbb{G}$, one index at a time. The replacement keeps the public keys, the access structure, the simulated sharing proof, and all opened group shares for $i \in U$ unchanged.

For each replaced index, a distinguisher between two adjacent games gives a DDH distinguisher: the reduction embeds the DDH tuple into $(g, \text{pk}_i, g^{s_i}, C_i^*)$,

where g^{s_i} is the corresponding group share in the GSS-on-group sharing. If the tuple is Diffie-Hellman, the ciphertext has the real distribution $\text{pk}_i^{s_i}$; if the tuple is random, the ciphertext is uniform in $\mathbb{G}$. Therefore, by a standard hybrid argument over the $m - |U|$ unopened indices,

$$|\Pr[E_1] - \Pr[E_2]| \leq (m - |U|) \cdot \text{Adv}_{\mathbb{G}}^{\text{ddh}}(\mathcal{B}_{\text{ddh}}) + \text{negl}(\lambda).$$

Game G_3 : replace unauthorized opened shares. In G_2 , all ciphertexts for $i \notin U$ are uniform and independent of b . The only remaining values that may depend on b are the decrypted group shares $\{S_i^* = g^{s_i}\}_{i \in U}$ returned to $\mathcal{A}$. Since U is unauthorized for $\mathbb{A}^*$, privacy of the underlying GSS-on-group scheme implies that these group shares can be replaced by a simulated unauthorized view independent of whether the shared secret is g^{x_0} or g^{x_1} . Thus

$$|\Pr[E_2] - \Pr[E_3]| \leq \text{Adv}_{\text{GrpGSS}}^{\text{priv}}(\mathcal{B}_{\text{gss}}) + \text{negl}(\lambda).$$

In G_3 , the adversary's entire view consists of simulated sharing proofs, uniform unopened ciphertexts, and simulated unauthorized opened group shares, all independent of b . Hence $\Pr[E_3] = 1/2$. Combining the above bounds gives

$$\text{Adv}_{\text{PVGSS}}^{\text{ind1}}(\mathcal{A}) \leq \text{Adv}_{\text{FS}}^{\text{zk}}(\mathcal{B}_{\text{zk}}) + (m - |U|) \cdot \text{Adv}_{\mathbb{G}}^{\text{ddh}}(\mathcal{B}_{\text{ddh}}) + \text{Adv}_{\text{GrpGSS}}^{\text{priv}}(\mathcal{B}_{\text{gss}}) + \text{negl}(\lambda),$$

which is negligible under the stated assumptions. $\square$

6 DEX Based on PVGSS

6.1 High-Level Overview

Based on the proposed PVGSS scheme, we design a DEX to allow exchangers to swap tokens fairly and simultaneously. In this research, we merely focus on ERC-20² token exchange. The DEX involves two roles: exchangers and watchers. Anyone can be exchangers and watchers. Each exchanger holds specific types of ERC-20 tokens, while multiple watchers collectively form a passive notary committee to address potential disputes. Take two exchangers, i.e., Alice and Bob, and n watchers $(W_1, \dots, W_n)$ as an example, the access structure is defined as:

$$\mathbb{A} = (2 \text{ of } (Alice, Bob, (t \text{ of } (W_1, W_2, \dots, W_n)))) \quad (1)$$

Based on Corollary 1, we can infer that the set $[Alice, Bob]$ is an authorized set. Apparently, if both Alice and Bob are honest, they can recover any secrets of PVGSS instance. If either exchanger is malicious, say Bob, Alice can recover the secret with help of at least t watchers. Due to the designed incentive mechanism, fairness does not rely on assuming that more than t watchers are honest. In particular, t can be set to 1: one responsive and economically accountable watcher is sufficient to resolve a dispute, while non-responsive or faulty watchers are penalized.

² <https://eips.ethereum.org/EIPS/eip-20>

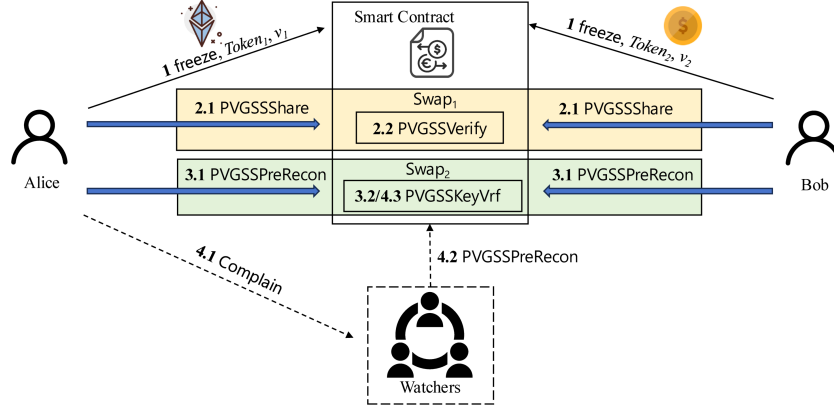


Fig. 9: Overview of the DEX between two exchangers (i.e., Alice and Bob) with passive watchers

The DEX operates under a commit-and-reveal paradigm. As shown by Figure 9, the DEX optimistically runs in two communication rounds for Alice and Bob. In the first round, swap_1 , each exchanger commits to a secret using PVGSSShare , where all the $n + 2$ entities are considered as shareholders. The correctness of the commitment is guaranteed by the PVGSSVerify algorithm. In the second round, swap_2 , each exchanger reveals its decrypted share using PVGSSPreRecon . The correctness of share decryption is ensured by PVGSSKeyVrf . Then, both Alice and Bob jointly recover each other's secrets using PVGSSRecon . In a pessimistic occasion where a player complains to the passive watchers, who will be involved to resolve dispute using PVGSSPreRecon .

6.2 Concrete Design

Ideal functionality. Abstracting away from implementation details, any reasonable decentralized exchange must satisfy three semantic requirements: (i) atomic execution of asset transfers, ensuring that either all swaps complete or none do; (ii) absence of unilateral advantage, meaning that no party can obtain the counterparty's asset without releasing its own; and (iii) public verifiability of outcomes, allowing any observer to determine whether a swap has succeeded or failed. We therefore define the ideal functionality $\mathcal{F}_{\text{DEX}}$ of a decentralized exchange by Figure 10. The functionality $\mathcal{F}_{\text{DEX}}$ is defined in the $\mathcal{F}_{\text{Ledger}}$ -hybrid model and relies on the ledger time and public state to enforce deadlines and asset transfers. Time is modeled via global deadlines $\Delta t_1 < \Delta t_2$, enforced by $\mathcal{F}_{\text{Ledger}}$. If the functionality outputs **success**, the exchange completes fairly; otherwise, the exchange is aborted. For notation, $x \in \{A, B\}$ denotes an exchanger, $\bar{x}$ denotes the counterparty, and $K_y^{(x)}$ denotes party y 's decrypted PVGSS share for exchanger x 's commitment.

Real-world implementation. Figure 11 describes the real-world protocol Π_{DEX} , which securely implements $\mathcal{F}_{\text{DEX}}$ in the $\mathcal{F}_{\text{Ledger}}$ -hybrid model. Each exe-

Ideal Functionality $\mathcal{F}_{\text{DEX}}$

The ideal functionality $\mathcal{F}_{\text{DEX}}$ operates in the $\mathcal{F}_{\text{Ledger}}$ -hybrid model. It is parameterized by time bounds $\Delta t_1 < \Delta t_2$, and interacts with $\mathcal{F}_{\text{Ledger}}$ to enforce time, store state, and manage asset escrow. Denote $\mathcal{A}$ as the adversary controlling corrupted parties. For each session id , it maintains registered keys, deposits, commitment flags, exchanger openings, complaints, and watcher opening sets. Upon activation, $\mathcal{F}_{\text{DEX}}$ behaves as follows:

- **Register:** Upon receiving $(\text{register}, \text{id}, \text{pid}, \text{pk}_y)$ from a party y , stores pk_y in its local state via $\mathcal{F}_{\text{Ledger}}$ and sends the public event to $\mathcal{A}$.
- **Deposit:** Upon receiving $(\text{freeze}, \text{id}, x, \text{Token}_x, v_x)$ from exchanger x , instructs $\mathcal{F}_{\text{Ledger}}$ to lock the tokens in escrow. Withdrawals are permitted only if the exchange either does not commence or terminates in **failure**.
- **Commit (swap₁):** Upon receiving $(\text{commit}, \text{id}, x)$ from exchanger x , records that x has committed and sends the public event to $\mathcal{A}$. The timer starts when both exchangers have committed.
- **Reveal (swap₂):** Upon receiving $(\text{reveal}, \text{id}, x)$ from exchanger x before Δt_1 , records that x has opened its commitment and sends the public event to $\mathcal{A}$.
- **Complain:** After Δt_1 , if exactly one exchanger x has not opened, allow the counterparty $\bar{x}$ to submit $(\text{complain}, \text{id}, x)$ and record a complaint against x .
- **Watcher opening:** After a valid complaint against x , upon receiving $(\text{arbitrate}, \text{id}, x, i)$ from an assigned watcher W_i before Δt_2 , records i in the watcher opening set for x and sends the public event to $\mathcal{A}$.
- **Determine:** Upon an explicit determine request:
 - If both exchangers have opened, instructs $\mathcal{F}_{\text{Ledger}}$ to transfer tokens and output **(success)**.
 - If exactly one exchanger has not opened but at least t assigned watchers have opened for that exchanger, does the same as above.
 - Otherwise, after Δt_2 , instructs $\mathcal{F}_{\text{Ledger}}$ to refund and output **(failure)**.

Fig. 10: The ideal functionality $\mathcal{F}_{\text{DEX}}$

cution instance is uniquely identified by a session id , which binds all application-level operations (**register**, **deposit**, etc.) to a single atomic exchange and prevents cross-session interference. The protocol involves two exchangers (Alice and Bob) and n watchers, who are randomly selected from a registered and staked pool for each id . A **state** variable tracks the lifecycle of the instance. Without loss of generality, Alice offers v_1 units of Token_1 in exchange for v_2 units of Token_2 .³

The concrete incentive and penalty policy is deferred to Appendix B, where we enumerate the successful and failed termination cases and the corresponding rewards or penalties.

³ We assume rational exchangers with access to external price oracles.

Real Protocol Π_{DEX}

Any two exchangers (Alice and Bob) leverage the same access structure by Equation (1). The real-world protocol Π_{DEX} is executed on smart contracts and implements $\mathcal{F}_{\text{DEX}}$ via PVGSS. The canonical leaf order is $(A, B, W_1, \dots, W_n)$, and every PVGSS public-key list and ciphertext vector follows this order. For exchanger $x \in \{A, B\}$, write $C^{(x)} = (C_A^{(x)}, C_B^{(x)}, C_1^{(x)}, \dots, C_n^{(x)})$ and let $K_y^{(x)}$ denote party y 's decrypted share of $C_y^{(x)}$.

- **Register:** Any entity registers its public key pk_y via $(\text{register}, \text{id}, \text{pid}, \text{pk}_y)$. The contract rejects invalid or identity group elements.
- **Deposit:** Alice and Bob freeze tokens via $(\text{freeze}, \text{id}, A, \text{Token}_A, v_A)$ and $(\text{freeze}, \text{id}, B, \text{Token}_B, v_B)$, respectively. If the session later terminates in **failure**, either party may reclaim its tokens via $(\text{withdraw}, \text{id})$.
- **Commit** (swap_1): Each exchanger $x \in \{A, B\}$ samples a secret s_x and invokes $(\text{swap}_1, \text{id}, x, C^{(x)}, \text{prf}_s^{(x)})$, where $(C^{(x)}, \text{prf}_s^{(x)}) \leftarrow \text{PVGSSShare}(s_x, \mathbb{A}, \text{pks})$ and $\text{pks} = (\text{pk}_A, \text{pk}_B, \text{pk}_1, \dots, \text{pk}_n)$. The application-level session identifier id , chain identifier, contract address, ordered public keys, and role label of x are recorded by the contract to prevent cross-session interference. The contract runs $\text{PVGSSVerify}(C^{(x)}, \text{prf}_s^{(x)}, \mathbb{A}, \text{pks})$; if it fails, the transaction is aborted.
- **Reveal** (swap_2): Exchanger x invokes $(\text{swap}_2, \text{id}, x, x, K_x^{(x)}, \text{prf}_{K_x}^{(x)})$, where $(K_x^{(x)}, \text{prf}_{K_x}^{(x)}) \leftarrow \text{PVGSSPreRecon}(C_x^{(x)}, \text{sk}_x)$. The contract checks $\text{PVGSSKeyVrf}(C_x^{(x)}, K_x^{(x)}, \text{prf}_{K_x}^{(x)}, \text{pk}_x)$; if valid, $K_x^{(x)}$ is stored. The counterparty $\bar{x}$ computes $K_{\bar{x}}^{(x)}$ locally and recovers $g^{s_x} \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q_x)$, where $Q_x = \{(x, K_x^{(x)}), (\bar{x}, K_{\bar{x}}^{(x)})\}$.
- **Complain:** If at time $t > \Delta t_1$ exchanger x has not revealed, the counterparty $\bar{x}$ may call $(\text{complain}, \text{id}, x)$, triggering a notification to all watchers to execute $(K_i^{(x)}, \text{prf}_{K_i}^{(x)}) \leftarrow \text{PVGSSPreRecon}(C_i^{(x)}, \text{sk}_i)$ and publish $(\text{swap}_2, \text{id}, x, i, K_i^{(x)}, \text{prf}_{K_i}^{(x)})$. The contract verifies each watcher response with $\text{PVGSSKeyVrf}(C_i^{(x)}, K_i^{(x)}, \text{prf}_{K_i}^{(x)}, \text{pk}_i)$. The complainant combines its locally computed share $K_{\bar{x}}^{(x)}$ with any t accepted watcher shares to reconstruct g^{s_x} .
- **Determine:** Any user may call $(\text{determine}, \text{id})$. The contract checks:
 - Both exchangers revealed their own openings $\Rightarrow$ **success**.
 - Exactly one exchanger did not reveal, and at least t valid watcher openings for the missing exchanger's commitment are stored $\Rightarrow$ **success**.
 - Otherwise, after $\Delta t_2 \Rightarrow$ set **failure** and refund.

Fig. 11: The real-world protocol Π_{DEX} for implementing $\mathcal{F}_{\text{DEX}}$

6.3 Security Properties

We prove in Appendix C that the real-world protocol Π_{DEX} UC-securely realizes the ideal functionality $\mathcal{F}_{\text{DEX}}$ and the DEX Π_{DEX} guarantees the security properties by Lemmas 4-10.

7 Conclusion

We introduced publicly verifiable generalized secret sharing (PVGSS), which combines expressive monotone access structures with non-interactive public accountability. The construction encrypts only the leaf shares of an underlying GSS instance and uses Schnorr-type Fiat-Shamir proofs together with GSS testing to certify that all encrypted shares are globally consistent with a single dealer secret. We instantiated the framework from recursive Shamir secret sharing and from LSSS, and proved correctness, verifiability of encrypted shares, verifiability of share decryption, and IND1-secrecy under explicit assumptions on DDH, the proof systems, and the privacy of the underlying GSS-on-group sharing. We also demonstrated how PVGSS can serve as a cryptographic building block for accountable blockchain protocols through a decentralized exchange. In this application, PVGSS enables verifiable commitment, verifiable secret release, and public arbitration under a flexible access structure. Several directions remain open. First, PVGSS can be applied to other systems that require both expressive authorization and public auditability, including randomness beacons, bridges, channels, or oracle networks. Second, it is useful to explore additional PVGSS constructions with smaller proofs, lower verification cost, or post-quantum assumptions. Finally, dynamic variants such as proactive PVGS, q-PVGSS and aggregatable PVGSS, analogous to proactive SS [58], q-PVSS [10] and aggregatable PVSS [63], may further broaden the applicability of publicly verifiable sharing under general access structures.

A Motivation of the Proposed DEX

Centralized exchanges (CEXs), such as Binance⁴ and Coinbase⁵, provide liquidity and usability but rely on central authorities, exposing users to breaches, mismanagement, or censorship and creating single points of failure. Decentralized exchanges (DEXs) emerged to address these issues, offering trustless trading where assets remain user-controlled and transactions secured by blockchain. DEXs inherently tackle the fair exchange problem, proven impossible without a trusted third party [60]. Hash time-locked contracts (HTLC)⁶ and adaptor signatures [39,42] enable atomic swaps [40,41], essential for DEXs. Yet, they lack efficient arbitration, causing delays or risks when parties refuse cooperation, e.g., withholding preimages or signatures. These methods emphasize fairness between traders but neglect broader incentives for liquidity providers (LPs) and arbitrageurs crucial to market growth [43]. Automated market maker (AMM) DEXs, such as Uniswap⁷, use liquidity pools and algorithms to set prices [44], simplifying trading and avoiding strict time-locks. However, LPs face permanent loss when pool prices shift, while traders encounter slippage in large trades.

⁴ <https://www.binance.com/en>

⁵ <https://www.coinbase.com>

⁶ https://en.bitcoin.it/wiki/Hash_Time_Locked_Contracts

⁷ <https://app.uniswap.org>

AMMs also suffer from front-running and miner extractable value (MEV) attacks [45,46], plus capital inefficiency requiring large liquidity to reduce slippage but yielding suboptimal returns [47]. Table 2 compares the different exchanges.

Table 2: Comparisons of different exchanges

Exchanges	Fairness provider	Arbitrageurs	No SPOF	Arbitration	No AMM slippage	Divergence Losslessness
CEX	CEX	N/A	✗	✗	✓	✓
HTLC-based	Miners	N/A	✓	✗	✓	✓
AMM-based	Smart contract	LPs	✓	✗	✗	✗
Ours	Smart contract	Watchers	✓	✓	✓	✓

B DEX Incentive Mechanism

A swap instance can be finalized as either **success** or **failure** by an explicit on-chain call to **determine** (proved by Lemma 5). Specifically, **success** can be determined before Δt_2 and **failure** can be determined only after Δt_2 . We assume that all entities are required to make stakes in the system; the number of swap instances a watcher can serve simultaneously is proportional to the amount of its stake. Figure 12 presents the mindmap to illustrate all result cases and the corresponding incentive policy for a swap instance.

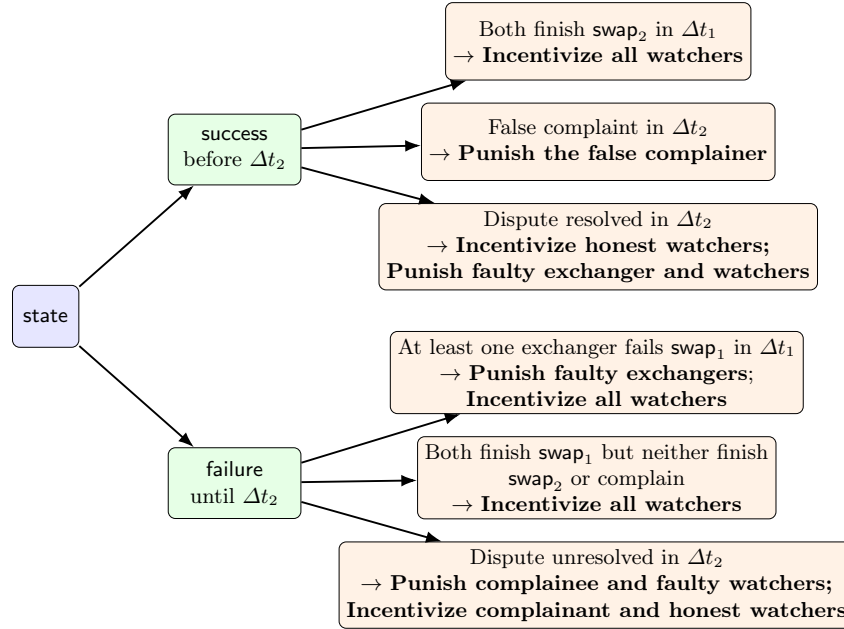


Fig. 12: The incentive and penalty mechanism

The **success** results include three cases. The first case is the optimistic scenario where both exchangers honestly invoke **swap**₁ and **swap**₂ before Δt_1 . In this case,

the watchers are not involved and each of them is rewarded with a small tip. The second case introduces the scenario where some exchanger may propose false complaint. In this case, the complainer will be punished. The third case shows the scenario where one exchanger does not execute `swap2` before Δt_2 . Then, the other exchanger notifies the n watchers to resolve the dispute. In this case, the faulty exchanger and non-responsive watchers are punished to reward the honest watchers who send `swap2` before Δt_2 .

The **failure** results also contain three cases. The first case means at least one exchanger gives up in Δt_1 . Hence, the faulty exchanger is punished and all watchers are rewarded. The second case means both exchangers negotiate to abort the swap instance before Δt_2 , and all watchers are rewarded. The third case is the exceptional scenario where an honest complaint is raised but remains unresolved due to insufficient honest watcher responses. We impose penalties on the complaine and non-responsive watchers to compensate the complainant and honest watchers.

C UC-Based Security of the DEX

This appendix proves that the concrete DEX protocol Π_{DEX} realizes the ideal functionality $\mathcal{F}_{\text{DEX}}$. The proof is centered on the protocol interfaces used by the implementation: `register`, `deposit`, `swap1`, `complain`, `swap2`, and `determine`. These interfaces correspond respectively to registration, asset escrow, commitment, complaint, reveal/arbitration, and settlement in $\mathcal{F}_{\text{DEX}}$.

Model and assumptions. We work in the $\mathcal{F}_{\text{Ledger}}$ -hybrid UC model. The ledger maintains an append-only public state and executes the DEX contract deterministically. A PPT adversary $\mathcal{A}$ statically corrupts parties, controls message scheduling, and observes all public events, but cannot alter the ledger state or contract code. The underlying PVGSS scheme satisfies correctness, public verifiability, and IND1-secrecy (Theorems 1–4); all group elements are validated; the proof systems used in PVGSSShare are simulation-extractable in the ROM; and the DLEQ proofs used in `swap2` are sound and adaptive zero-knowledge.

The DEX fairness argument uses economic accountability rather than an honest-watcher threshold. Every watcher response is checked on-chain by `PVGSSKeyVrf`. Missing, invalid, or premature arbitration is attributable and is penalized by the incentive mechanism in Appendix B. Thus the implementation may set $t = 1$: one responsive watcher resolves a valid complaint, while faulty or non-responsive watchers lose stake.

Theorem 5 (UC Security of Π_{DEX}). *Under the assumptions above, for any PPT real-world adversary $\mathcal{A}$ interacting with Π_{DEX} and $\mathcal{F}_{\text{Ledger}}$, there exists a PPT simulator $\mathcal{S}$ interacting with $\mathcal{F}_{\text{DEX}}$ and $\mathcal{F}_{\text{Ledger}}$ such that for every PPT environment $\mathcal{Z}$,*

$$\text{EXEC}_{\Pi_{\text{DEX}}, \mathcal{A}, \mathcal{Z}} \approx_c \text{EXEC}_{\mathcal{F}_{\text{DEX}}, \mathcal{S}, \mathcal{Z}}.$$

Here $\approx_c$ denotes computational indistinguishability.

Proof. We construct $\mathcal{S}$ by emulating the ledger and the DEX contract for $\mathcal{A}$, while forwarding the semantic effect of each accepted contract call to $\mathcal{F}_{\text{DEX}}$.

Registration and deposits. For honest parties, $\mathcal{S}$ samples $(\text{sk}_i, \text{pk}_i) \leftarrow \text{PVGSSSetup}(\lambda)$ and registers pk_i . For corrupted parties, it forwards adversarially chosen public keys and deposits only when they satisfy the same validation and escrow checks as the contract.

Commitment via swap₁. For an honest exchanger x , $\mathcal{S}$ samples s_x , runs $(C^{(x)}, \text{prf}_s^{(x)}) \leftarrow \text{PVGSSShare}(s_x, \mathbb{A}, \text{pks})$, records the ciphertext vector, and forwards $(\text{commit}, \text{id}, x)$ to $\mathcal{F}_{\text{DEX}}$. For a corrupted exchanger, it first runs the contract verification $\text{PVGSSVerify}(C^{(x)}, \text{prf}_s^{(x)}, \mathbb{A}, \text{pks})$. Rejected commitments are rejected in the simulation. For accepted commitments, simulation-extractability yields witnesses consistent with the statement, so $\mathcal{S}$ can bind the real swap₁ event to the ideal commitment event.

Complaint and reveal via swap₂. A valid complain call after Δt_1 is forwarded as a complaint in $\mathcal{F}_{\text{DEX}}$. For an honest reveal or watcher arbitration, $\mathcal{S}$ computes $(K_y^{(x)}, \text{prf}_{K_y}^{(x)}) \leftarrow \text{PVGSSPreRecon}(C_y^{(x)}, \text{sk}_y)$ from the recorded ciphertext. For a corrupted submission, it runs the same PVGSSKeyVrf check as the contract. A valid exchanger reveal is forwarded as $(\text{reveal}, \text{id}, x)$; a valid watcher response to an active complaint is forwarded as $(\text{arbitrate}, \text{id}, x, i)$; invalid or premature responses are rejected.

Settlement via determine. When determine is invoked, $\mathcal{S}$ queries $\mathcal{F}_{\text{DEX}}$ for the ideal outcome and writes the corresponding success or failure transition to the simulated ledger, including the same transfers, refunds, rewards, and slashing decisions.

Simulation consistency. The simulator accepts and rejects the same DEX messages as the real contract, except with negligible proof-system error.

The theorem follows because honest commitments and reveals are generated by the real PVGSS algorithms, while every accepted corrupted commitment or reveal is backed by an extractable or publicly verifiable witness (Theorems 2 and 3). Before reveal or arbitration, IND1-secrecy hides honest exchangers' secrets from unauthorized sets. Therefore the environment sees the same ledger events, the same contract acceptance behavior, and computationally indistinguishable cryptographic messages in the real and ideal executions. This gives the claimed UC realization by a standard hybrid argument. $\square$

C.1 DEX Properties Implied by the UC Realization

Lemma 4 (Decentralization). *Protocol Π_{DEX} does not rely on a trusted third-party operator.*

Proof. Registration, escrow, verification, arbitration, and settlement are performed by the public contract and PVGSS verification algorithms. No operator holds a special key or decides the outcome off-chain. $\square$

Lemma 5 (Termination). *Every execution of Π_{DEX} can be finalized within a bounded time once an honest finalization transaction is delivered by the ledger.*

Proof. The contract enforces $\Delta t_1 < \Delta t_2$. If the required secrets are available before Δt_1 , determine finalizes success; otherwise, after Δt_2 , any delivered finalization transaction finalizes failure and refunds the escrowed assets. $\square$

Lemma 6 (Optimistic Fairness). *Protocol Π_{DEX} guarantees fairness: an honest exchanger either obtains the correct counterparty assets or can finalize a refund, assuming ledger liveness and the incentive mechanism of Appendix B.*

Proof. swap_1 binds each exchanger to publicly verifiable PVGSS shares before any transfer can occur. If both exchangers reveal before Δt_1 , PVGSS correctness reconstructs the secrets and the contract finalizes success. If one exchanger withholds its reveal, a valid complaint activates watcher arbitration; with $t = 1$, any one responsive watcher can submit a verified swap_2 response and complete the exchange. Invalid commitments and decrypted shares are rejected by PVGSSVerify and PVGSSKeyVrf, and missing, invalid, or premature watcher actions are attributable and slashable. If arbitration is not completed by Δt_2 , the contract finalizes failure and refunds the escrowed assets. Hence an honest exchanger either receives the counterparty assets or recovers its own assets; this conclusion uses ledger liveness and incentive compatibility, not an assumption that more than t watchers are honest. $\square$

Lemma 7 (Accountable Exchangers). *Any dishonest behavior by an exchanger is publicly detectable.*

Proof. PVGSSVerify checks exchanger commitments in swap_1 , and PVGSSKeyVrf checks decrypted shares in swap_2 . Thus malformed commitments, invalid reveals, and missed mandatory actions are visible on-chain. $\square$

Lemma 8 (Accountable Arbitration). *All arbitration actions performed by watchers are publicly verifiable and accountable.*

Proof. After a complaint, each watcher response consists of a decrypted share and a DLEQ proof generated by PVGSSPreRecon. The contract verifies the response with PVGSSKeyVrf, so invalid, missing, or premature arbitration is publicly attributable. $\square$

Lemma 9 (Optimistic and Stateless Arbitration). *Protocol Π_{DEX} supports optimistic and stateless arbitration.*

Proof. When both exchangers reveal on time, settlement uses only exchanger swap_2 messages and no watcher is invoked. Upon a complaint, watchers compute responses from on-chain ciphertexts and their long-term secret keys, so they need no per-swap local state. $\square$

Lemma 10 (Incentive Compatibility). *Π_{DEX} admits slashing-based mechanisms that make valid watcher arbitration the rational strategy, allowing the protocol to set $t = 1$.*

Proof. By Lemmas 7 and 8, outcome-affecting deviations are detectable. Let B be the maximum deviation benefit, G the gas cost of honest arbitration, R the reward for a valid watcher response, and D the slashable stake. If $D > B$ and $R > G$, a watcher maximizes payoff by answering valid complaints and avoiding invalid, missing, or premature responses. Therefore one economically accountable watcher is sufficient for dispute resolution, allowing $t = 1$. $\square$

Claim 1 (Resistance of DoS attack) *DoS attempts can delay progress only until Δt_2 . Missing or invalid required actions are attributable on-chain and are penalized under the stake bounds of Lemma 10.* $\square$

Claim 2 (Resistance of Collusion/Coercion attack) *Public verification and slashing make collusive watcher behavior attributable and economically irrational. Even with $t = 1$, a faulty watcher cannot make an honest exchanger lose escrowed assets: the complaint is either resolved by a valid response or finalized as failure after Δt_2 .* $\square$

Claim 3 (Resistance of Sybil attack) *Watcher influence is proportional to locked, slashable stake. Splitting stake across Sybil identities does not increase total influence and keeps large-scale deviation costly.* $\square$

Claim 4 (Resistance of Rushing attack) *Premature watcher responses are rejected or penalized because arbitration is valid only after an active complaint. Thus rushing does not change the settlement outcome for honest exchangers.* $\square$

D Experimental Evaluation

We implement our PVGSS scheme using Golang (off-chain) and Solidity (on-chain). The method in [20] supports only boolean formula-based access structures; therefore, we adopt the approach proposed in [56] to transform a general access structure into a LSSS matrix. To make on-chain and off-chain operations compatible, we choose BN128 as the asymmetric elliptic curve group, which is officially supported by Ethereum. The off-chain benchmarks of PVGSS schemes are executed on an Apple M4 with 16 GB RAM running macOS 26.3 and Golang 1.25.4. The smart contract codes are deployed and tested on a local Ethereum test network.

D.1 Off-Chain PVGSS Costs

We choose the access structure designed for the DEX (cf. Equation (1)) to evaluate the performance of the PVGSS scheme, where $t = n/2 + 1$ and n ranges from 100 to 1100. Figure 13-16 show the off-chain computational overheads of the PVGSS schemes.

Figure 13 reports the cost of PVGSSShare. Both constructions grow super-linearly because the sharing procedure generates one encrypted group share for each participant and constructs the corresponding Schnorr-type proof material.

The Shamir SS-based curve is slightly higher at large n , mainly because its implementation evaluates the threshold polynomial over the full participant set, whereas the LSSS-based implementation shares along the DEX access tree and then maps the resulting LSSS shares to group commitments.

Figure 14 reports the cost of PVGSSVerify. Verification first checks the NIZK equations for all encrypted shares and then invokes one GSS testing routine. The exact interpolation test evaluates the required values directly with precomputed Lagrange coefficients, rather than recovering full polynomial coefficients by generic Gaussian elimination. The dual-code test reuses precomputed barycentric weights for the fixed evaluation domain and reconstructs internal-node secrets by Lagrange interpolation at zero. The dual-code curves are therefore higher in this implementation, because the dual relation is checked over the relevant evaluation domain in addition to the per-share NIZK checks. These optimizations preserve the interpolation and dual-code verification relations defined in Definition 9.

Figure 15 reports n executions of PVGSSKeyVrf. The curves are close to linear because each decrypted share is verified independently by one DLEQ proof. The LSSS-based and Shamir SS-based variants almost overlap, since key verification depends only on (C_i, K_i, pk_i) and the DLEQ proof, not on how the access structure generated the original share.

Figure 16 reports online PVGSSRecon after preparing public reconstruction coefficients. For the LSSS-based construction, the reconstruction weights are computed from the fixed access structure and authorized set before the timed online phase; the online algorithm only performs the resulting group linear combination. For the Shamir SS-based construction, the current implementation recursively reconstructs along the access tree and recomputes interpolation coefficients on the selected path, which explains its steeper growth. This precomputation does not affect security, because the coefficients depend only on public access-control information and not on the secret or decrypted shares.

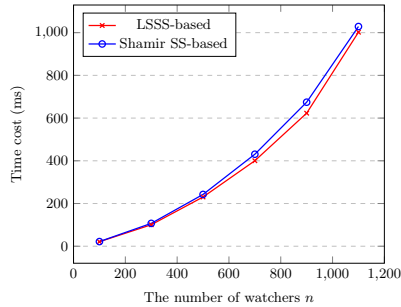


Fig. 13: Cost of PVGSSShare

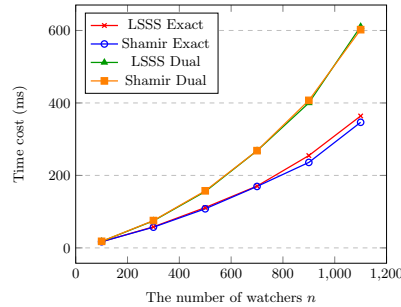


Fig. 14: Cost of PVGSSVerify

D.2 On-Chain DEX Costs

Further, we evaluate the gas consumption of the DEX. The on-chain prototype reports costs for $t = 1$ and n ranging from 1 to 10. We first introduce

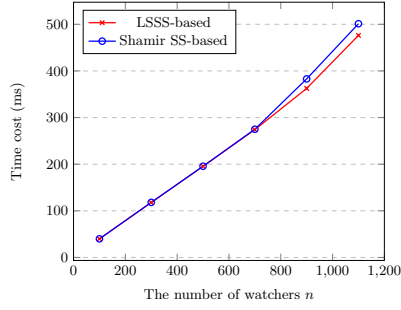


Fig. 15: Cost of n times of PVGSSKeyVrf

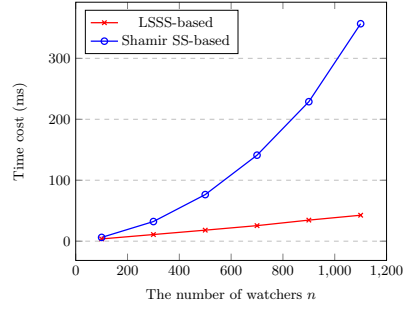


Fig. 16: Online PVGSSRecon with an authorized set size $t + 1$

the cost in the normal case, where both exchangers honestly abide by the protocol. In swap_1 , calculation of PVGSSVerify and storage of $\{C_i\}$ are included. The swap_2 , which invokes PVGSSKeyVrf, roughly costs a constant amount of gas (about 140000) for each exchanger. The PVGSSKeyVrf algorithm is the same for both LSSS-based and Shamir SS-based PVGSS, hence they cost equal gas consumption. Figure 17 depicts the gas consumption of the $\text{swap}_1 + \text{swap}_2$ on smart contract. The four curves separate exact interpolation and dual-code GSS testing for both LSSS-based and Shamir SS-based PVGSS, matching the off-chain verification variants. The on-chain implementation keeps the same verification relations but avoids generic polynomial-coefficient recovery: the exact test evaluates the interpolation polynomial directly at zero and at the remaining child points, while the dual-code test computes the fixed-domain dual weights using factorial denominators. Then, we depict the gas cost in exceptional scenarios where arbitration procedures are required. Particularly, an exchanger calls `complain`, then each watcher invokes swap_2 and finally anyone can execute `determine`. Figure 18 shows the gas cost of functionalities of `complain`, `determine` and swap_2 .

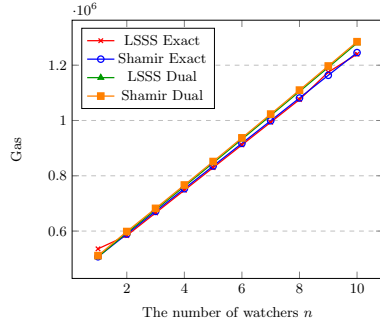


Fig. 17: On-chain gas cost of $\text{swap}_1 + \text{swap}_2$

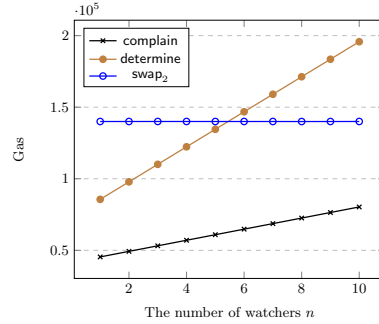


Fig. 18: On-chain cost of `complain`, `determine`, swap_2

As of the time of writing (November 25, 2025), the median gas price is 0.206 gwei, and the price of ETH is 2,958 USD.⁸ Table 3 presents the monetary cost

⁸ <https://etherscan.io/gastracker>

Table 3: Monetary cost of the on-chain operations

Operation	Gas cost	USD
register	89985	\$0.055
freeze	195114	\$0.119
swap ₁	$n=1$ 367121	\$0.224
	$n=5$ 695237	\$0.424
	$n=9$ 1023451	\$0.624
swap ₂	139999	\$0.085
total , i.e., swap ₁ + swap ₂ (Shamir SS-based)	$n=1$ 507120	\$0.309
	$n=5$ 835236	\$0.509
	$n=9$ 1163438	\$0.709
complain	$n=1$ 45399	\$0.028
	$n=5$ 60911	\$0.037
	$n=9$ 76423	\$0.047
determine	$n=1$ 85604	\$0.052
	$n=5$ 134552	\$0.082
	$n=9$ 183500	\$0.112

of each operation on the smart contract. For convenience, only the Shamir SS-based PVGSS is evaluated. The gray cells highlight the optimistic case, where the cost for each exchanger is only \$0.309/\$0.509/\$0.709 for $n = 1/5/9$ and $t = 1$. If watchers are involved in the pessimistic case, the complaint cost is less than 0.047 USD, and each watcher incurs a cost of approximately 0.085 USD (as swap₂ requires).

E Decisional Diffie-Hellman (DDH) Assumption

The Decisional Diffie-Hellman (DDH) [11] assumption states that (g, g^a, g^b, g^{ab}) and (g, g^a, g^b, g^c) are computationally indistinguishable, where $a, b, c \xleftarrow{R} \mathbb{Z}_q$ are chosen uniformly at random. Formally, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ in distinguishing between the two distributions is negligible:

$$|\Pr [\mathcal{A}(g, g^a, g^b, g^{ab}) = 1] - \Pr [\mathcal{A}(g, g^a, g^b, g^c) = 1]| \leq \text{negl}(\lambda),$$

where λ is the security parameter, and $\text{negl}(\lambda)$ denotes a negligible function in λ .

F Formal Definition of Security Properties for PVGSS

Definition 11 (Correctness). *The PVGSS is correct if for every access structure $\mathbb{A}$, every labelled authorized index set I_Q , and each $s \in \mathbb{Z}_q$,*

$$\Pr \left[\begin{array}{l} \text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}, \{\text{pk}_i\}) = 1 \wedge \\ \forall i \in I_Q : \\ \text{PVGSSKeyVrf}(C_i, S_i, \text{prf}_{S_i}, \text{pk}_i) = 1 \wedge \\ S' = g^s \end{array} \middle| \begin{array}{l} \forall i \in [|\mathbb{A}|], (\text{sk}_i, \text{pk}_i) \leftarrow \text{PVGSSSetup}(\lambda); \\ (\{C_i\}, \text{prf}_s) \leftarrow \text{PVGSSShare}(s, \mathbb{A}, \{\text{pk}_i\}) \\ \forall i \in I_Q, (S_i, \text{prf}_{S_i}) \leftarrow \text{PVGSSPreRecon}(C_i, \text{sk}_i) \\ Q \leftarrow \{(i, S_i)\}_{i \in I_Q}; \\ (S') \leftarrow \text{PVGSSRecon}(\mathbb{A}, Q); \end{array} \right] = 1$$

Definition 12 (Verifiability of Encrypted Shares). *The PVGSS satisfies verifiability of encrypted shares if for every access structure $\mathbb{A}$ and every PPT $\mathcal{A}$, where $m = |\mathbb{A}|$ and $\text{Valid}_{\mathbb{A}}(s, \mathbf{s})$ denotes that $\mathbf{s} = (s_1, \dots, s_m)$ is a valid GSS sharing of s under $\mathbb{A}$,*

$$\Pr \left[\begin{array}{l} \text{PVGSSVerify}(\{C_i\}, \text{prf}_s, \mathbb{A}, \{\text{pk}_i\}) = 1 \wedge \\ \nexists (s, \mathbf{s}) \in \mathbb{Z}_q \times \mathbb{Z}_q^m \text{ s.t.} \\ \text{Valid}_{\mathbb{A}}(s, \mathbf{s}) \wedge \forall i \in [m], C_i = \text{pk}_i^{s_i} \end{array} \middle| \begin{array}{l} \forall i \in [m], (\text{sk}_i, \text{pk}_i) \leftarrow \text{PVGSSSetup}(\lambda), \\ (\{C_i\}, \text{prf}_s) \leftarrow \mathcal{A}(\mathbb{A}, \{\text{pk}_i\}) \end{array} \right] \leq \text{negl}(\lambda)$$

Definition 13 (Verifiability of Share Decryption). *The PVGSS satisfies verifiability of share decryption if for every PPT $\mathcal{A}$ and every polynomially bounded number N of registered keys,*

$$\Pr \left[\begin{array}{l} \text{PVGSSKeyVrf}(C, S, \text{prf}_S, \text{pk}) = 1 \wedge \\ \nexists x \in \mathbb{Z}_q^* \text{ s.t. } \text{pk} = g^x \text{ and } C = S^x \end{array} \middle| \begin{array}{l} \forall i \in [N], (\text{sk}_i, \text{pk}_i) \leftarrow \text{PVGSSSetup}(\lambda), \\ (\text{pk}, C, S, \text{prf}_S) \leftarrow \mathcal{A}(\{\text{pk}_i\}), \text{pk} \in \{\text{pk}_i\} \end{array} \right] \leq \text{negl}(\lambda)$$

Definition 14 (IND1-secrecy). *IND1-secrecy denotes indistinguishability of shared secrets against an adversary controlling only an unauthorized set. PVGSS achieves **IND1-secrecy** if every PPT adversary $\mathcal{A}$ wins the following game with only negligible advantage.*

1. $\mathcal{A}$ outputs a challenge access structure $\mathbb{A}^*$.
2. The challenger runs PVGSSSetup for each leaf of $\mathbb{A}^*$ and publishes the ordered public keys $\{\text{pk}_i\}_{i \in [|\mathbb{A}^*|]}$.
3. $\mathcal{A}$ outputs two candidate secrets $x_0, x_1 \in \mathbb{Z}_q$ and an index set $U \subseteq [|\mathbb{A}^*|]$ whose labels do not satisfy $\mathbb{A}^*$.
4. The challenger samples $b \xleftarrow{R} \{0, 1\}$ and invokes $(\{C_i^*\}, \text{prf}_{x_b}) \leftarrow \text{PVGSSShare}(x_b, \mathbb{A}^*, \{\text{pk}_i\})$. It sends $(\{C_i^*\}, \text{prf}_{x_b})$ to $\mathcal{A}$.
5. The adversary may request decrypted shares for indices in U ; for each such query i , the challenger returns $(S_i^*, \text{prf}_{S_i^*}) \leftarrow \text{PVGSSPreRecon}(C_i^*, \text{sk}_i)$.
6. $\mathcal{A}$ then outputs a guess $b' \in \{0, 1\}$.

The advantage of $\mathcal{A}$ is defined as $|\Pr[b = b'] - \frac{1}{2}|$.

References

1. Shamir, A. How to share a secret. *Comm. of the ACM*, 1979, 22(11), pp. 612-613.
2. Das, S., Yurek, T., Xiang, Z., Miller, A., Kokoris-Kogias, L. and Ren, L. Practical asynchronous distributed key generation. In *S&P*, 2022, pp. 2518-2534.
3. Choi, K., Manoj, A. and Bonneau, J. Sok: Distributed randomness beacons. In *S&P*, 2023, pp. 75-92.
4. Syta, E., Jovanovic, P., Kogias, E.K., Gailly, N., Gasser, L., Khoffi, I., Fischer, M.J. and Ford, B. Scalable bias-resistant distributed randomness. In *S&P*, 2017, pp. 444-460.
5. Schindler, P., Judmayer, A., Stifter, N. and Weippl, E., 2020, May. Hydrand: Efficient continuous distributed randomness. In 2020 IEEE Symposium on Security and Privacy (SP) (pp. 73-89). IEEE.
6. Sahai, A., and Waters, B. Fuzzy identity-based encryption. In *Eurocrypt*, 2005, pp. 457-473.
7. Karchmer, M. and Wigderson, A. On span programs. In *Annual Structure in Complexity Theory Conference*, 1993, pp. 102-111.
8. Feldman, P. A practical scheme for non-interactive verifiable secret sharing. In *FOCS*, 1987, pp. 427-438.
9. Zhang, Z., Li, W., Guo, Y., Shi, K., Chow, S.S., Liu, X. and Dong, J. Fast RS-IOP Multivariate Polynomial Commitments and Verifiable Secret Sharing. In *USENIX Security*, 2024, pp. 3187-3204.
10. Cascudo, I., and David, B. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In *Eurocrypt*, 2024, pp. 216-248.
11. Cascudo, I., and David, B. SCRAPE: Scalable randomness attested by public entities. In *ACNS*, 2017, pp. 537-556.
12. Kate, A., Zaverucha, G.M. and Goldberg, I., Constant-size commitments to polynomials and their applications. In *Asiacrypt*, 2010, pp. 177-194.
13. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S. and Stern, G. Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In *Crypto*, 2023, pp. 39-70.
14. Stadler, M. Publicly verifiable secret sharing. In *Eurocrypt*, 1996, pp. 190-199.
15. Ito, M., Saito, A. and Nishizeki, T. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan*, 1989, 72(9), pp. 56-64.
16. Benaloh, J., and Leichter, J. Generalized secret sharing and monotone functions. In *Crypto*, 1990.
17. Garg, S., Jain, A., Mukherjee, P., Sinha, R., Wang, M. and Zhang, Y. Cryptography with weights: MPC, encryption and signatures. In *CRYPTO*, 2023, pp. 295-327.
18. Cao, B., Xiao, S., Shi, L., Wang, T., Chen, J., Wang, J., Ling, X., Xu, H., Zhang, S. and Liu, E. Web 3.0: A survey on the architectures, enabling technologies, applications, and challenges. *IEEE Communications Surveys & Tutorials*, 2025.
19. Bethencourt, J., Sahai, A., and Waters, B. Ciphertext-Policy Attribute-Based Encryption. In *S&P*, 2007, pp. 321-334.
20. Lewko, A., and Waters, B. Decentralizing attribute-based encryption. In *Eurocrypt*, 2011, pp. 568-588.
21. Lin, C., Hu, H., Chang, C.C. and Tang, S. A publicly verifiable multi-secret sharing scheme with outsourcing secret reconstruction. *IEEE Access*, 2018, 6, pp. 70666-70673.
22. Beimel, A. Secret-Sharing Schemes for General Access Structures: An Introduction. *Cryptology ePrint Archive*. 2025.

23. Schnorr, C.P., 1990. Efficient identification and signatures for smart cards. In *Crypto*, 1990, pp. 239-252.
24. Drăgan, C.C. and Țiplea, F.L., . Distributive weighted threshold secret sharing schemes. *Information sciences*, 2016, 339, pp. 85-97.
25. Beimel, A., Tassa, T., and Weinreb, E. Characterizing ideal weighted threshold secret sharing. In *TCC*, 2005, pp. 600-619.
26. Chen, Q., Tang, C. and Lin, Z. Efficient explicit constructions of multipartite secret sharing schemes. *IEEE TIT*, 2021, 68(1), pp. 601-631.
27. Tassa, T., and Dyn, N. Multipartite secret sharing by bivariate interpolation. *JoC*, 2009, 22(2), pp. 227-258.
28. Simmons, G.J., 1988, August. How to (really) share a secret. In *Eurocrypt*, 1988, pp. 390-448.
29. Damgård, I. On Σ -protocols. *Lecture Notes, University of Aarhus, Department for Computer Science*, 2002.
30. Fiat, A., and Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In *Eurocrypt*, 1986, pp. 186-194.
31. Tassa, T. Hierarchical threshold secret sharing. *JoC*, 2007, 20(2), pp. 237-264.
32. Chattopadhyay, A.K., Saha, S., Nag, A. and Nandi, S. Secret sharing: A comprehensive survey, taxonomy and applications. *Computer Science Review*, 2024, 51, 100608.
33. Beimel, A. Secret-sharing schemes: A survey. In *ICC*, 2011, pp. 11-46.
34. Cascudo, I., David, B., Garms, L., and Konring, A. YOLO YOSO: fast and simple encryption and secret sharing in the YOSO model. In *Asiacrypt*, 2022, pp. 651-680.
35. Cascudo, I., and David, B. ALBATROSS: publicly attestable batched randomness based on secret sharing. In *Asiacrypt*, 2020, pp. 311-341.
36. Fujisaki, E., and Okamoto, T. A practical and provably secure scheme for publicly verifiable secret sharing and its applications. In *Eurocrypt*, 1998, pp. 32-46.
37. Schoenmakers, B. A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic. In *Crypto*, 1999, pp. 148-164.
38. Ruiz, A., and Villar, J. L. Publicly verifiable secret sharing from Paillier's cryptosystem. In *SAC*, 2005.
39. Erwig, A., Faust, S., Hostáková, K., Maitra, M. and Riahi, S. Two-party adaptor signatures from identification schemes. In *PKC*, 2021, pp. 451-480.
40. Thyagarajan, S.A., Malavolta, G. and Moreno-Sanchez, P. Universal atomic swaps: Secure exchange of coins across all blockchains. In *SECP*, 2022, pp. 1299-1316.
41. Zhang, L., Kan, H., Qiu, F. and Hao, F. A Publicly Verifiable Optimistic Fair Exchange Protocol Using Decentralized CPABE. *CJ*, 2024, 67(3), pp. 1017-1029.
42. Aumayr, L., Ersoy, O., Erwig, A., Faust, S., Hostáková, K., Maffei, M., Moreno-Sanchez, P. and Riahi, S. Generalized channels from limited blockchain scripts and adaptor signatures. In *Asiacrypt*, 2021, pp. 635-664.
43. Augusto, A., Belchior, R., Correia, M., Vasconcelos, A., Zhang, L. and Hardjono, T. Sok: Security and privacy of blockchain interoperability. In *SECP*, 2024, pp. 3840-3865.
44. Xu, J., Paruch, K., Cousaert, S. and Feng, Y. Sok: Decentralized exchanges (dex) with automated market maker (amm) protocols. *ACM Comp. Surv.*, 2023, 55(11), pp. 1-50.
45. Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L. and Juels, A. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *SECP*, 2020, pp. 910-927.
46. Baum, C., David, B. and Frederiksen, T.K. P2DEX: privacy-preserving decentralized cryptocurrency exchange. In *ACNS*, 2021, pp. 163-194.

47. Heimbach, L., Schertenleib, E. and Wattenhofer, R. Risks and returns of uniswap v3 liquidity providers. In *AFT*, 2022, pp. 89-101.
48. Park, S., Bastani, O. and Kim, T. ACon2: Adaptive Conformal Consensus for Provable Blockchain Oracles. In *USENIX Security*, 2023, pp. 3313-3330.
49. Xie, T., Zhang, J., Cheng, Z., Zhang, F., Zhang, Y., Jia, Y., Boneh, D. and Song, D. zkbridge: Trustless cross-chain bridges made practical. In *CCS*, 2022, pp. 3003-3017.
50. Li, Y., Liu, H. and Tan, Y. POLYBRIDGE: A crosschain bridge for heterogeneous blockchains. In *ICBC*, 2022, pp. 1-2.
51. Dziembowski, S., Faust, S. and Hostáková, K. General state channel networks. In *CCS*, 2018, pp. 949-966.
52. Tyagi, N., Arun, A., Freitag, C., Wahby, R., Bonneau, J. and Mazières, D. Riggs: Decentralized sealed-bid auctions. In *CCS*, 2023, pp.1227–1241.
53. Benhamouda, F., Gentry, C., Gorbunov, S., Halevi, S., Krawczyk, H., Lin, C., Rabin, T. and Reyzin, L. Can a public blockchain keep a secret?. In *TCC*, 2020, pp. 260-290.
54. Goyal, V., Kothapalli, A., Masserova, E., Parno, B. and Song, Y. Storing and retrieving secrets on a blockchain. In *PKC*, 2022, pp. 252-282.
55. Cerulli, A., Connolly, A., Neven, G., Preiss, F.S. and Shoup, V. vetkeys: How a blockchain can keep many secrets. *Cryptology ePrint Archive*. 2023.
56. Liu, Z., Cao, Z. and Wong, D.S. Efficient generation of linear secret sharing scheme matrices from threshold access trees. *Cryptology ePrint Archive*. 2010
57. Chaum, D., & Pedersen, T. P. Wallet databases with observers. In *CRYPTO*, 1992, pp. 89-105.
58. Bagherpour, B., 2022. An efficient publicly verifiable and proactive secret sharing scheme. *Adv. in Math. of Comm.*, 16(4), pp. 671-691.
59. Alon, N., Galil, Z. and Yung, M. Efficient dynamic-resharing “verifiable secret sharing” against mobile adversary. In *EPA*, 1995, pp. 523-537.
60. Pagnia, H. and Gärtner, F.C. On the impossibility of fair exchange without a trusted third party. Technical Report TUD-BS-1999-02, Darmstadt University of Technology, Department of Computer Science, Darmstadt, Germany. 1999.
61. Galil, Z. and Yung, M. Partitioned encryption and achieving simultaneity by partitioning. *Infor. Proc. Letters*, 1987, 26(2), pp.81-88.
62. Canetti, R. Security and composition of multiparty cryptographic protocols. *Journal of Cryptography*, 2000, 13(1), pp.143-202.
63. Bacho, R. and Loss, J. Adaptively secure (aggregatable) pvss and application to distributed randomness beacons. In *CCS*, 2023, pp. 1791-1804.